UHASSELT
KNOWLEDGE IN ACTION

Maastricht University

Faculteit Wetenschappen
School voor Informatietechnologie

master in de informatica

Masterthesis

Recursive Query Optimisation

Wolf Van Wesemael

Scriptie ingediend tot het behalen van de graad van master in de informatica

PROMOTOR :

Prof. dr. Stijn VANSUMMEREN

BEGELEIDER :

De heer Olaf VAN BYLEN

2024
2025

De transnationale Universiteit Limburg is een uniek samenwerkingsverband van twee universiteiten in twee landen: de Universiteit Hasselt en Maastricht University.

UHASSELT
KNOWLEDGE IN ACTION

www.uhasselt.be
Universiteit Hasselt
Campus Hasselt:
Martelarenlaan 42 | 3500 Hasselt
Campus Diepenbeek:
Agoralaan Gebouw D | 3590 Diepenbeek

Maastricht University

Faculteit Wetenschappen
School voor Informatietechnologie

master in de informatica

Masterthesis

Recursive Query Optimisation

Wolf Van Wesemael

Scriptie ingediend tot het behalen van de graad van master in de informatica

PROMOTOR :

Prof. dr. Stijn VANSUMMEREN

BEGELEIDER :

De heer Olaf VAN BYLEN

Acknowledgements

First and foremost, I would like to express my gratitude to my promoter Prof. Dr. Vansummeren for his guidance and support during this endeavor. His clear and direct feedback has greatly enhanced the quality of my work, and his insight and knowledge have also been invaluable in understanding the necessary subject matter.

I would also like to thank my advisor Olaf Van Bylen for his constructive feedback and insightful suggestions, which helped refine my work.

A sincere thank you to all my professors during my education who have defined my academic growth, as well as to my friends who also had a part in this process.

Lastly, I also want to thank my family for giving me the opportunity to receive this education and for giving me the necessary support when needed.

Summary

0.1 Introduction

In the current data-driven world, drawing meaningful insights from complex data is vital for applications across many domains. Often, recursive reasoning is of key importance in this process. Traditional SQL and naive query execution strategies often fall short of what is desired by companies and researchers in terms of efficiency and expressiveness. A popular language for expressing these recursive queries is Datalog. It lends itself well to expressing these queries because of its rule-based formulation. This thesis focuses on improving the efficiency of recursive queries expressed in Datalog. There exists a plethora of techniques for improving the efficiency of Datalog evaluation, but we focus specifically on a query rewriting technique called 'magic sets transformation' which hopes to decrease the number of tuples derived during evaluation. We benchmark an implementation of this technique and some other more basic techniques as well as some other Datalog engines so we can accurately identify the influence of this technique.

0.2 Datalog

Datalog is a declarative logic programming language. It's primarily used for querying databases and executing logical inference based on rules and the facts in the database. A Datalog program consists of rules that define how new data should be deduced from what is already present in the database. These rules consist of predicates. A predicate consists of a database table name and that table's arguments. One predicate describes the new tuple to be derived. This predicate is called the rule head. The rule body consists of other predicates, of which the join describes the conditions that the derived tuple must abide by. We also make a distinction when it comes to predicates. There are intensional database predicates (IDB predicates) which are predicates for which we derive tuples during our Datalog program. Aside from those, there are also extensional database predicates (EDB predicates) which are the source database tables that do not change during Datalog evaluation.

There are three ways to define Datalog semantics: model-theoretic semantics, fixpoint-theoretic semantics, and proof-theoretic semantics. Though each is very different from the other, they can often be used interchangeably. In model-theoretic semantics, each rule is treated as a first-order logic constraint. In this semantics, the result of a Datalog program is the minimal model. An unofficial definition of the minimal model: the set of tuples extending the input database with everything we can derive from it without introducing new constants. In fixpoint-theoretic semantics, the focus is more on how a result is computed. Vital to this semantics is the immediate consequence operator T_P which derives new facts from existing ones. This operator is used iteratively until no more new facts are derived. At this point, the resulting set of tuples is called the least fixpoint and is equivalent to the minimal model of the previous semantics. In proof-theoretic semantics, the result of a Datalog program is the set of all atoms that can be proven using the source database instance and the rules of the Datalog program. Proofs can be generated in two ways. The first way is bottom-up where we derive more and more proofs by repeatedly applying rules and the second way is top-down where we try to prove a given output by trying to prove other atoms from which our given output can be proven.

An option to extend the expressiveness of core Datalog is adding the possibility for negation of predicates in a rule body (this is called Datalog$^\neg$). However, negation can bring problems, so we have to add extra semantics. There are two versions of Datalog with negation. The first is called semipositive Datalog$^\neg$. With this semantics, negation can only be applied to EDB predicates and every variable in a negated

predicate must also appear in a non-negated predicate. The second version of Datalog with negation is called stratified Datalog$^\neg$. With this stratified semantics, a Datalog program is actually a sequence of different semipositive Datalog$^\neg$ programs. With the result of every program in the sequence serving as EDB predicates for the following programs in the sequence. This sequence of subprograms is called a stratification. There are two rules to decide what stratum a predicate belongs to and consequently which subprograms will contain which rules. The first rule states that the predicate of a rule head must appear in the same or a later stratum than any of the non-negated predicates from the corresponding rule body. The second rule states that the predicate of the rule head must appear in a later stratum than any of the negated predicates from the corresponding rule body. The result for a stratified Datalog$^\neg$ program can be computed by computing the result for every stratum in order. The stratification of a program can be computed through the use of a finite precedence graph, where every predicate from a rule body points towards the predicate in the corresponding rule head with an optional label for negated predicates. If no loops are present with an edge that has a label for negation, then a stratification is possible and it consists of the sequence of topologically sorted strongly connected components.

Another option for extending the expressiveness of core Datalog is aggregation. An aggregate variable consists of an aggregate operator and its arguments. For aggregation, we also need to introduce extra conditions. An aggregate variable can only occur in the head of a rule, and none of its arguments can be aggregates themselves. Furthermore, each of the arguments of the aggregate must appear in a predicate in the rule body and cannot appear in the rule head as a non-aggregate variable. Just like with negation, we need extra semantics to avoid any problems aggregation can cause. Consequently, we again rely on stratification. Stratification for aggregation works similarly to how it works with negation, but there is one extra rule: the predicate of the rule head must appear in a later stratum than any of the predicates from the corresponding rule body that have a variable that occurs as an argument of an aggregate variable in the rule head. Computing a stratification works analogously to how it works with stratified Datalog$^\neg$ with the addition of a label for aggregation that should not appear in a loop in the finite precedence graph.

Even more Datalog extensions are covered in the full thesis, but we will not discuss them in this summary.

0.3 Recursive Query Processing

There are multiple methods for evaluating recursive queries. We make a distinction between bottom-up and top-down methods. The first category works by applying the rules of the program repeatedly to derive new tuples to compute the minimal model. The second category pushes the selection criteria from the query into the computation to reduce the number of necessary derivations.

The simplest bottom-up evaluation algorithm we can use is the repeated application of the immediate consequence operator of the fixpoint-theoretic semantics. Each iteration, we derive what tuples we can from all tuples we already have until we reach an iteration where nothing new is derived. This is often called the naïve method. A major shortcoming of this method is the fact that every iteration it repeats all derivations that have already been executed during previous iterations.

The semi-naïve method is an improvement on the naïve method that aims to address the major shortcoming described above. It aims to reduce the redundant derivations by applying the idea that new tuples can only be derived from a rule when a tuple is present that was derived in the last iteration.

Now we take a look at top-down evaluation methods which also focus on decreasing the number of derivations. However, instead of limiting the number of repeat derivations, top-down focuses more on decreasing the number of derivations that are not necessary for computing the result of a query. The technique we will be looking at is called Query-Subquery (QSQ) and in it, the query provides information on potential values for the query predicate, which is called binding information. We can then unify this information with the rules that have the same predicate in the rule head. The binding information used in the rule head can now be used in the corresponding variables in the rule body. Through passing along binding information to predicates, we obtain subqueries. Solving these subqueries can provide binding information for variables that were not bound yet, which can then be passed along to subsequent predicates. This process keeps going until a final result is computed.

Binding information is usually materialized as a set of tuples. To more easily pass binding information through rules, a concept called adornment is used. Adorned rules are like normal rules with the addition

that every predicate has an extra string that indicates which of its arguments are bound. So if a predicate has the adornment string bf, then that means the first argument is bound and the second is not. Adornment for a rule is computed by determining an adornment for the rule head and then applying the adornment of the variables on corresponding variables in the predicates of the rule body. Any unbound variables in a predicate from the rule body should then also be considered bound when computing the adornment of subsequent predicates in the rule body.

Semi-naïve evaluation limits the number of redundant repeated derivations, while Query-Subquery limits the number of derivations unnecessary for computing a result to a given query. Ideally, we combine both of these techniques to get the best of both worlds. Magic sets rewriting is a program rewriting technique that attempts this. The idea behind magic sets is that the binding information used throughout the program can be represented as relations and the computation of these binding predicates as Datalog rules. Every adorned rule now gets an input relation for the predicate in the rule head, which contains the binding information for that adorned rule. Furthermore, every predicate in a rule body is now preceded by a supplementary predicate containing all the binding information for that predicate. Every supplementary relation also has its own rule for computing what tuples should be present in it. Likewise for the input relations if applicable since some input relations start with data (from the query), but do not gain new tuples over the course of evaluation (when no subqueries with the same adornment occur).

There is a plethora of optimizations you can make while using the magic sets rewrite technique. For example, for an adorned rule, it is possible to replace the rule body with just the last predicate and its corresponding supplementary relation, since this supplementary relation will contain all binding information computed by the preceding predicates in the rule body. However, not all optimizations are so straightforward. Other important factors for the efficiency of the program resulting from the rewrite are: the ordering of predicates in the rule bodies, ensuring the selectivity provided by the computed supplementary relations is worth the computational cost, and determining which variables should be bound within each predicate in order to minimize the size and consequently the cost of materializing the supplementary relations. These aspects are called sideways information passing strategies (SIPS) and should be optimized so that the supplementary relations containing binding information provide maximum selectivity for minimum computation costs.

0.4 An Overview of Datalog Systems

Datalog has been utilized in many different fields, leading to the creation of many different Datalog engines, such as: Graspan [24], Soufflé [22], BigDatalog [19], RecStep [23], LogicBlox [18], Socialite [20], and bddbddb [21]. These Datalog engines share foundational principles but still differ in many ways.

One such thing they differ in is the query plan generation for the rules in the Datalog program. An engine can choose between a dynamic approach, where each iteration the engine attempts to compute an optimal query plan, or the engine chooses a static approach, where a single query plan is computed for all iterations ahead of time. Each approach has its pros and cons. The dynamic approach necessitates more computation during evaluation in exchange for an optimal plan, while the static approach saves on computation costs, but the computed plan might degrade over the evaluation. Plan optimization is also important, and it can be rather complex because statistics for IDB relations do not exist and are difficult to estimate.

When performing semi-naïve evaluation of rules, the difference operation is used intensively when computing the set of new tuples for each iteration. To avoid any bottlenecks, this operation must be optimized. Some engines do this using a specialized operator, while other engines opt for specialized data structures.

How to perform parallel data processing is also an important design choice that requires attention. There are various possible architectures. One such family of architectures is the shared-nothing architectures, which rely on a large network of independent processors/machines connected via a fast communication network. An important choice to make for these architectures is which data partitioning strategy to use. However, the design choices go even further. The subset of attributes to partition by is also something to consider. Some programs even allow for parallel evaluation where no IDB relations need to be communicated across the network after the original broadcast of EDB tuples, given the correct subset of attributes is chosen to partition by. Unfortunately, this program trait is not systematically

determinable, though several conditions have been found that, if present, confirm this trait.

Another aspect critical to parallel data processing is whether processors work synchronously or asynchronously. In synchronous execution, machines wait at the end of every iteration for every other machine to finish (usually for communication of relations). However, this is an obvious risk of bottlenecks. In asynchronous execution, machines can be in different iterations. This removes the risk of bottlenecks, but sadly not all programs lend themselves to asynchronous execution. It might also lead to the reintroduction of redundant derivations because of accessing stale data from other machines. Neither method seems to work definitively better than the other.

Shared-memory Datalog architectures make up another prominent family of architectures. Here, all machines share resources such as main memory and disk. The main concern for this architecture is handling conflicts over shared data structures. There are various ways to tackle this problem. Some engines opt for a simple execution algorithm combined with more intricate specialized data structures, while other engines might rely on an underlying database management system, et cetera.

Whether a Datalog program should be compiled or interpreted is another design aspect that needs to be considered. The efficiency of compiled programs is nice, but some information only becomes known during the evaluation of a Datalog program, so for this reason, the amount of optimizations one can apply is limited when compiling a Datalog program.

While most Datalog engines simply use the classic row-oriented data representation, some engines opt for specialized data structures, trading in the advantages of classic row-oriented data representation for extra efficiency in terms of space and/or execution time. Some of these data structures include: binary decision diagrams, tail-nested tables, tries, bit matrices, and more.

Just like with data structures, the choice of indexes is an important one. An engine can opt for a more classic index such as hash-tables or B-trees, or an engine can make use of specialized indexes. It is even a possibility to defer the creation of indexes until runtime.

Aside from all the design choices already mentioned which can impact the efficiency of a Datalog program, there are also optimization techniques that can be used to influence the efficiency. There are techniques such as the magic set transformation mentioned previously, techniques for reducing the number of iterations or strata in a program, techniques for pushing aggregation into recursion to reduce the amount of tuples derived during evaluation, and more.

0.5 Implementation Architecture

To evaluate the influence of the magic sets rewriting technique on the efficiency of Datalog evaluation, a custom implementation was developed. The custom implementation supports naïve Datalog evaluation, semi-naïve Datalog evaluation, and semi-naïve Datalog evaluation combined with the magic sets rewriting technique.

The implementation was developed using the programming language Rust. There are several reasons for this design choice. First, Rust and its standard libraries have comprehensive documentation. This often also applies to community-made libraries. Second, Rust has a large central repository containing many community-produced libraries. Third, Rust has excellent performance, code safety, and ease of use through the use of a plethora of techniques. Last, Rust provides extensive support for concurrency.

The implementation is built upon Apache DataFusion, which is an extensible, open-source, in-memory, columnar data management system and query engine. Apache DataFusion provides us with basic implementations of a plethora of components we need to build our custom Datalog engine. Furthermore, it also provides a lot of functionality to construct and optimize logical plans for our queries. On top of that, Apache DataFusion provides other performance-oriented features and utilizes the parallelism provided by Rust's Tokio library.

For our data, we work with Parquet files, which use a columnar binary format. Using Apache Parquet has various benefits: Parquet files save space by utilizing compression, they store data types and schema metadata, and they are quite efficient for reading large datasets.

The custom implementation expects various different input files. The pre-processing to prepare the files portraying Datalog programs and relationship tables is mostly done through Python scripts because of Python's ease of use.

For the implementation logic of the custom Datalog engine, we refer the user to the full thesis as this would be too in-depth for this summary.

0.6 Comparison

Our benchmarks were run on five different engines wherever possible, these being: custom naïve implementation, custom semi-naïve implementation, custom semi-naïve implementation combined with magic sets rewriting, RecStep, and Soufflé. By comparing our custom implementation with RecStep and Soufflé, our comparison gains broader relevance because of the different design philosophies of these Datalog engines.

As benchmark datalog programs, we use a subset of the programs described in the sixth section of the RecStep paper [42], together with a variant of one of these programs, which can be found in 'Modern Datalog Engines' [40] by Bas Ketsman and Paraschos Koutris. These programs originate from two fields: graph analytics and static program analysis. The benchmarks were run with various datasets. The specific datasets changed based on the current program's characteristics.

From our benchmark times we notice many patterns, some more predictable than others. For example, the overhead paired with techniques other than the naïve evaluation can cause them to be slower if the dataset is too small. However, execution times in these cases are all small enough so that this is a non-issue. When looking more at how the semi-naïve evaluation holds up, we notice that our custom implementation seems to perform slightly better than RecStep for smaller datasets, while RecStep seems to scale slightly better. Soufflé, on the other hand, seems to be more inconsistent when it comes to performance compared to the custom semi-naïve implementation. This is attributed to its compilation-focused design, which seems to give varying results based on the program and dataset.

If we shift our attention to evaluating magic set results, we see varying levels of success. For some benchmarks, magic sets transformation provides a significant decrease in execution time, while for other benchmarks, the magic sets technique instead makes the evaluation take longer to finish than normal semi-naïve evaluation. To understand these measurements more, we also investigated the total number of deduced tuples during evaluation. This seems to follow the same pattern as the execution time. When the magic sets transformation results in a program that derives fewer tuples than normal semi-naïve evaluation, then the execution time will be lower than with semi-naïve evaluation. On the contrary, if the resulting program derives more tuples than normal semi-naïve evaluation, then the evaluation will take longer to finish.

We had hoped for a more consistent and positive influence from the magic sets technique, but the result is not completely surprising. The efficiency of the program resulting from magic sets is dependent on the sideways information passing strategies employed. The custom implementation of the magic sets transformation is quite a simple one and thus not a lot of attention was paid to this. Consequently, the measurement results could look a lot different when combined with a more intricate implementation of the magic sets transformation. Another important aspect is the binding information provided by the query. If the set of provided bindings is smaller, then it will follow that the resulting execution time will also be smaller, since the bindings are more selective.

0.7 Conclusions

We can conclude from our benchmark results that semi-naïve evaluation engines outperform naïve evaluation engines for any dataset size for which optimization is interesting and that the magic set rewriting technique is heavily dependent on sideways information passing strategies and the size of the set of query bindings.

This thesis was not able to prove that the magic set rewriting technique combined with semi-naïve evaluation is definitively superior to normal semi-naïve evaluation of a Datalog program. Further research into the topic is necessary. This research can explore different subjects, such as: sideways information passing strategies and their influence, possible integration of the magic sets rewriting technique into existing engines, and the behavior of the magic sets rewriting technique when combined with more complex Datalog variants.

Samenvatting

0.8 Introductie

In de huidige datagestuurde wereld is het verkrijgen van zinvolle inzichten uit complexe gegevens van vitaal belang voor toepassingen in vele domeinen. Vaak is recursief redeneren van groot belang in dit proces. Traditionele SQL en naïeve query-uitvoeringsstrategieën voldoen vaak niet aan de wensen van bedrijven en onderzoekers op het gebied van efficiëntie en expressiviteit. Een populaire taal om deze recursieve queries uit te drukken is Datalog. Het leent zich goed voor het uitdrukken van deze queries vanwege de regelgebaseerde formulering. Deze thesis richt zich op het verbeteren van de efficiëntie van recursieve queries uitgedrukt in Datalog. Er bestaan verschillende technieken om de evaluatie van Datalog efficiënter te maken, maar wij richten ons specifiek op een techniek voor het herschrijven van queries die 'magic sets transformatie' wordt genoemd. We benchmarken een implementatie van deze techniek en enkele andere meer basistechnieken, evenals enkele andere Datalog engines, zodat we de invloed van deze techniek nauwkeurig kunnen vaststellen.

0.9 Datalog

Datalog is een declaratieve logische programmeertaal. Het wordt voornamelijk gebruikt voor het bevragen van databases en het uitvoeren van logisch redeneren op basis van regels en de feiten in de database. Een Datalog programma bestaat uit regels die definiëren hoe nieuwe gegevens moeten worden afgeleid uit wat al aanwezig is in de database. Deze regels bestaan uit predikaten. Een predikaat bestaat uit de naam van een databasetabel en de argumenten van die tabel. Eén predicaat beschrijft de nieuwe tupel die moet worden afgeleid. Dit predicaat wordt de rule head genoemd. De rule body bestaat uit andere predikaten, waarvan de join de voorwaarden beschrijft waaraan de afgeleide tupel moet voldoen. We maken ook onderscheid in predikaten. Er zijn intensionele databasepredicaten (IDB predicaten), dat zijn predikaten waarvoor we tuples afleiden tijdens ons Datalog-programma. Daarnaast zijn er ook extensionele databasepredikaten (EDB predicaten), dit zijn de tabellen van de brondatabase die niet veranderen tijdens de evaluatie van het Datalog programma.

Er zijn drie manieren om Datalog semantiek te definiëren: model-theoretische semantiek, fixpoint-theoretische semantiek en bewijs-theoretische semantiek. Hoewel ze van elkaar verschillen, kunnen ze vaak door elkaar gebruikt worden. In model-theoretische semantiek wordt elke regel behandeld als een eerste-orde logica beperking. In deze semantiek is het resultaat van een Datalog programma het minimale model. Een onofficiële definitie van het minimale model: de verzameling tupels die de invoerdatabase uitbreidt met alles wat we ervan kunnen afleiden zonder nieuwe constanten te introduceren. In de fixpoint-theoretische semantiek ligt de nadruk meer op hoe een resultaat wordt berekend. Van vitaal belang voor deze semantiek is de immediate consequence operator T_P die nieuwe feiten afleidt uit bestaande feiten. Deze operator wordt iteratief gebruikt totdat er geen nieuwe feiten meer worden afgeleid. Op dit punt wordt de resulterende verzameling van tupels het least fixpoint genoemd en is gelijkwaardig aan het minimale model van de vorige semantiek. In de bewijstheoretische semantiek is het resultaat van een Datalog programma de verzameling van alle atomen die bewezen kunnen worden met behulp van de bron database instantie en de regels van het Datalog programma. Bewijzen kunnen op twee manieren worden gegenereerd. De eerste manier is bottom-up waarbij we meer en meer bewijzen afleiden door herhaaldelijk regels toe te passen en de tweede manier is top-down waarbij we een gegeven output proberen te bewijzen door te proberen andere atomen te bewijzen waaruit onze gegeven output kan worden bewezen.

Een optie om de uitdrukkingskracht van kern Datalog uit te breiden is het toevoegen van de mogelijkheid

tot negatie van predikaten in een rule body (dit heet Datalog$^\neg$). Negatie kan echter problemen opleveren, dus moeten we extra semantiek toevoegen. Er zijn twee versies van Datalog met negatie. De eerste heet semipositief Datalog$^\neg$. Met deze semantiek kan negatie alleen worden toegepast op EDB predikaten en bovendien moet elke variabele in een negatie predikaat ook voorkomen in een niet-negatie predikaat. De tweede versie van Datalog met negatie wordt gestratificeerd Datalog$^\neg$ genoemd. Met deze gestratificeerde semantiek is een Datalog programma eigenlijk een opeenvolging van verschillende semipositieve Datalog$^\neg$ programma's. Het resultaat van elk programma in de reeks dient als EDB-predikaat voor de volgende programma's in de reeks. Deze opeenvolging van subprogramma's wordt een stratificatie genoemd. Er zijn twee richtlijnen om te bepalen tot welk stratum een predikaat behoort en dus welke subprogramma's welke regels zullen bevatten. De eerste richtlijn stelt dat het predikaat van een rule head in hetzelfde of een later stratum moet voorkomen dan elk van de niet-negatie predikaten uit het overeenkomstige rule body. De tweede richtlijn stelt dat het predikaat van een rule head in een later stratum moet voorkomen dan elk van de negatie predikaten uit het corresponderende rule body. Het resultaat van een gestratificeerd Datalog$^\neg$ programma kan berekend worden door het resultaat voor elk stratum in volgorde te berekenen. De stratificatie van een programma kan worden berekend met behulp van een finite precedence graph, waarin elk predikaat uit een regel naar het predikaat in de corresponderende rule head wijst met een optioneel label voor negatie predikaten. Als er geen lussen aanwezig zijn met een edge die een label voor negatie heeft, dan is er een stratificatie mogelijk die bestaat uit de opeenvolging van topologisch gesorteerde sterk verbonden componenten.

Een andere optie om de uitdrukkingskracht van kern datalog uit te breiden is aggregatie. Een aggregaatvariabele bestaat uit een aggregaatoperator en zijn argumenten. Voor aggregatie moeten we ook extra voorwaarden introduceren. Een aggregaatvariabele kan alleen voorkomen in de rule head en geen van de argumenten kan zelf een aggregaat zijn. Bovendien moet elk van de argumenten van het aggregaat voorkomen in een predikaat in het rule body en kan het niet voorkomen in het rule head als een niet-aggregaatvariabele. Net als bij negatie hebben we extra semantiek nodig om problemen te vermijden die aggregatie kan veroorzaken. Daarom vertrouwen we opnieuw op stratificatie. Stratificatie voor aggregatie werkt op dezelfde manier als bij negatie, maar er is één extra regel: het predikaat van de rule head moet in een later stratum voorkomen dan elk van de predikaten uit het overeenkomstige rule body die een variabele hebben die voorkomt als argument van een aggregaatvariabele in de rule head. Het berekenen van een stratificatie werkt analoog aan hoe het werkt met gestratificeerde Datalog$^\neg$ met de toevoeging van een label voor aggregatie die niet mag voorkomen in een lus in de finite precedence graph.

Nog meer uitbreidingen op Datalog worden behandeld in de volledige thesis, maar we zullen ze niet bespreken in deze samenvatting.

0.10 Recursieve Verwerking van Queries

Er zijn meerdere methoden om recursieve queries te evalueren. We maken een onderscheid tussen bottom-up en top-down methoden. De eerste categorie werkt door de regels van het programma herhaaldelijk toe te passen om nieuwe tupels af te leiden om het minimale model te berekenen. De tweede categorie duwt de selectiecriteria van de query in de berekening om het aantal noodzakelijke afleidingen te verminderen.

Het eenvoudigste bottom-up evaluatiealgoritme dat we kunnen gebruiken is de herhaalde toepassing van de immediate consequence operator van de fixpunt-theoretische semantiek. Bij elke iteratie leiden we de tupels af die we kunnen afleiden van alle tupels die we al hebben, totdat we een iteratie bereiken waarin niets nieuws wordt afgeleid. Dit wordt vaak de naïeve methode genoemd. Een grote tekortkoming van deze methode is het feit dat in elke iteratie alle afleidingen worden herhaald die al tijdens eerdere iteraties zijn uitgevoerd.

De semi-naïeve methode is een verbetering op de naïeve methode die de hierboven beschreven tekortkoming wil aanpakken. Het doel is om het aantal overbodige afleidingen te verminderen door het idee toe te passen dat nieuwe tupels alleen van een regel kunnen worden afgeleid als er een tupel aanwezig is die in de laatste iteratie is afgeleid.

Nu kijken we naar top-down evaluatiemethoden die zich ook richten op het verminderen van het aantal afleidingen. Maar in plaats van het aantal herhaalde afleidingen te beperken, richt top-down zich meer op het verminderen van het aantal afleidingen die niet nodig zijn voor het berekenen van het resultaat van een query. De techniek die we zullen bekijken heet Query-Subquery (QSQ) en hierbij geeft de query

informatie over mogelijke waarden voor het query predikaat, wat bindingsinformatie wordt genoemd. We kunnen deze informatie dan verenigen met de regels die hetzelfde predikaat in de rule head hebben. De bindingsinformatie die gebruikt wordt in de rule head kan nu gebruikt worden in de corresponderende variabelen in de regel. Door bindingsinformatie door te geven aan predikaten, verkrijgen we subqueries. Het oplossen van deze subqueries kan bindingsinformatie opleveren voor variabelen die nog niet gebonden waren, die dan kan worden doorgegeven aan volgende predikaten. Dit proces gaat door totdat een eindresultaat is berekend.

Bindingsinformatie wordt meestal gematerialiseerd als een set tuples. Om bindingsinformatie gemakkelijker door regels te laten lopen, wordt een concept genaamd adornment gebruikt. Adorned regels zijn zoals normale regels met de toevoeging dat elk predikaat een extra string heeft die aangeeft welke van zijn argumenten gebonden zijn. Dus als een predikaat de adornment string bf heeft, dan betekent dit dat het eerste argument gebonden is en het tweede niet. De adornment van een regel wordt berekend door een adornment voor de rule head te bepalen en dan de adornment van de variabelen op overeenkomstige variabelen in de predikaten van de regel toe te passen. Alle ongebonden variabelen in een predikaat van de regel moeten dan ook als gebonden worden beschouwd bij het berekenen van de adornment van volgende predikaten in de regel.

Semi-naïeve evaluatie beperkt het aantal overbodige herhaalde afleidingen, terwijl Query-Subquery het aantal afleidingen beperkt dat onnodig is voor het berekenen van een resultaat voor een gegeven query. Idealiter combineren we beide technieken om het beste van beide werelden te krijgen. Magic sets rewriting is een techniek voor het herschrijven van programma's die dit probeert. Het idee achter magic sets is dat de bindende informatie die doorheen het hele programma wordt gebruikt, kan worden weergegeven als relaties en de berekening van deze bindende predikaten als Datalog regels. Elke adorned regel krijgt nu een invoerrelatie voor het predikaat in de rule head, die de bindingsinformatie voor die adorned regel bevat. Bovendien wordt elk predikaat in een regel nu voorafgegaan door een supplementair predikaat dat alle bindende informatie voor dat predikaat bevat. Elke supplementaire relatie heeft ook zijn eigen regel om te berekenen welke tupels erin moeten voorkomen. Hetzelfde geldt voor de invoerrelaties indien van toepassing, omdat sommige invoerrelaties beginnen met gegevens (van de query), maar geen nieuwe tupels krijgen in de loop van de evaluatie (als er geen subqueries met dezelfde adornment voorkomen).

Er zijn veel optimalisaties die je kunt maken bij het gebruik van de magic sets rewrite techniek. Bijvoorbeeld, voor een adorned regel is het mogelijk om het rule body te vervangen door alleen het laatste predikaat en de bijbehorende supplementaire relatie, omdat deze supplementaire relatie alle bindingsinformatie bevat die is berekend door de voorgaande predikaten in het rule body. Niet alle optimalisaties zijn echter zo eenvoudig. Andere belangrijke factoren voor de efficiëntie van het programma dat het resultaat is van het herschrijven zijn: de volgorde van de predikaten in de regels, ervoor zorgen dat de selectiviteit die de berekende supplementaire relaties bieden de berekeningskosten waard is en bepalen welke variabelen binnen elk predikaat moeten worden gebonden om de grootte en dus de kosten van het materialiseren van de supplementaire relaties te minimaliseren. Deze aspecten worden sideways information passing strategies (SIPS) genoemd en moeten worden geoptimaliseerd zodat de supplementaire relaties met bindingsinformatie maximale selectiviteit bieden tegen minimale berekeningskosten.

0.11 Een Overzicht van Datalogsystemen

Datalog wordt op veel verschillende gebieden gebruikt, wat heeft geleid tot de creatie van veel verschillende Datalog engines, zoals: Graspan [24], Soufflé [22], BigDatalog [19], RecStep [23], LogicBlox [18], Socialite [20], en bddbddb [21]. Deze Datalog engines hebben dezelfde basisprincipes, maar verschillen toch op veel manieren.

Zo verschillen ze bijvoorbeeld in het genereren van een queryplan voor de regels in het Datalog programma. Een engine kan kiezen tussen een dynamische aanpak, waarbij de engine elke iteratie een optimaal queryplan probeert te berekenen, of de engine kiest voor een statische aanpak, waarbij één queryplan voor alle iteraties van tevoren wordt berekend. Elke aanpak heeft voor- en nadelen. De dynamische aanpak vereist meer rekenwerk tijdens de evaluatie in ruil voor een optimaal plan, terwijl de statische aanpak bespaart op rekenkosten, maar het berekende plan kan degraderen tijdens de evaluatie. Planoptimalisatie is ook belangrijk en kan vrij complex zijn omdat statistieken voor IDB-relaties niet bestaan en moeilijk te voorspellen zijn.

Bij het uitvoeren van semi-naïeve evaluatie van regels wordt de verschiloperatie intensief gebruikt bij

het berekenen van de set nieuwe tupels voor elke iteratie. Om bottlenecks te voorkomen, moet deze bewerking worden geoptimaliseerd. Sommige engines doen dit met een gespecialiseerde operator, terwijl andere engines kiezen voor gespecialiseerde gegevensstructuren.

De manier waarop parallelle gegevensverwerking wordt uitgevoerd, is ook een belangrijke ontwerpkeuze die aandacht vereist. Er zijn verschillende architecturen mogelijk. Eén zo'n familie van architecturen zijn de shared-nothing architecturen, die vertrouwen op een groot netwerk van onafhankelijke processoren/machines die verbonden zijn via een snel communicatienetwerk. Een belangrijke keuze die gemaakt moet worden voor deze architecturen, is welke strategie voor het partitioneren van gegevens gebruikt moet worden. De ontwerpkeuzes gaan echter nog verder. De subset van attributen om te partitioneren is ook iets om te overwegen. Sommige programma's staan zelfs parallelle evaluatie toe waarbij geen IDB relaties over het netwerk gecommuniceerd hoeven te worden na de oorspronkelijke broadcast van EDB tupels, mits de juiste subset van attributen wordt gekozen om op te partitioneren. Helaas is deze programma-eigenschap niet systematisch vast te stellen, hoewel er verschillende voorwaarden zijn gevonden die, indien aanwezig, deze eigenschap bevestigen.

Een ander aspect dat cruciaal is voor parallelle gegevensverwerking is of processoren synchroon of asynchroon werken. Bij synchrone uitvoering wachten machines aan het einde van elke iteratie tot elke andere machine klaar is (meestal voor communicatie van relaties). Dit is echter een duidelijk risico op bottlenecks. Bij asynchrone uitvoering kunnen machines in verschillende iteraties zitten. Dit neemt het risico op bottlenecks weg, maar helaas lenen niet alle programma's zich tot asynchrone uitvoering. Het kan ook leiden tot de herintroductie van overbodige afleidingen vanwege toegang tot verouderde gegevens van andere machines. Geen van beide methoden lijkt definitief beter te werken dan de andere.

Shared-memory Datalog architecturen vormen een andere prominente familie van architecturen. Hier delen alle machines bronnen zoals hoofdgeheugen en schijf. De grootste zorg voor deze architectuur is het omgaan met conflicten over gedeelde gegevensstructuren. Er zijn verschillende manieren om dit probleem aan te pakken. Sommige engines kiezen voor een eenvoudig uitvoeringsalgoritme gecombineerd met meer ingewikkelde gespecialiseerde gegevensstructuren, terwijl andere engines kunnen vertrouwen op een onderliggend databasemanagementsysteem, enzovoort.

Of een Datalog programma gecompileerd of geïnterpreteerd moet worden is een ander ontwerpaspect dat overwogen moet worden. De efficiëntie van gecompileerde programma's is prettig, maar sommige informatie wordt pas bekend tijdens de evaluatie van een Datalog programma. Om deze reden is de hoeveelheid optimalisaties die men kan toepassen beperkt bij het compileren van een Datalog programma.

Terwijl de meeste Datalog engines gewoon de klassieke rijgeoriënteerde gegevensrepresentatie gebruiken, kiezen sommige engines voor gespecialiseerde gegevensstructuren, waarbij de voordelen van de klassieke rijgeoriënteerde gegevensrepresentatie worden ingeruild voor extra efficiëntie in termen van ruimte en/of uitvoeringstijd. Enkele van deze gegevensstructuren zijn: binaire beslissingsdiagrammen, tail-nested tabellen, tries, bitmatrices en meer.

Net als bij gegevensstructuren is de keuze van indexen belangrijk. Een engine kan kiezen voor een meer klassieke index zoals hash-tabellen of B-trees, of een engine kan gebruikmaken van gespecialiseerde indexen. Het is zelfs mogelijk om de creatie van indexen uit te stellen tot runtime.

Naast alle ontwerpkeuzes die al genoemd zijn en die de efficiëntie van een Datalog programma kunnen beïnvloeden, zijn er ook optimalisatietechnieken die gebruikt kunnen worden om de efficiëntie te beïnvloeden. Er zijn technieken zoals de eerder genoemde 'magic sets transformation', technieken om het aantal iteraties of strata in een programma te verminderen, technieken om aggregatie in recursie te duwen om het aantal afgeleide tuples tijdens evaluatie te verminderen, en meer.

0.12 Implementatie Architectuur

Om de invloed van de magic sets rewriting techniek op de efficiëntie van Datalog evaluatie te evalueren, werd een zelfgemaakte implementatie ontwikkeld. De zelfgemaakte implementatie ondersteunt naïeve Datalog evaluatie, semi-naïeve Datalog evaluatie en semi-naïeve Datalog evaluatie in combinatie met de magic sets rewriting techniek.

De implementatie is ontwikkeld met de programmeertaal Rust. Er zijn verschillende redenen voor deze ontwerpkeuze. Ten eerste hebben Rust en de standaardbibliotheken uitgebreide documentatie. Dit geldt vaak ook voor bibliotheken geproduceerd door de Rust gemeenschap. Ten tweede heeft Rust een grote

centrale repository die veel gemeenschapsgeproduceerde bibliotheken bevat. Ten derde heeft Rust uitstekende prestaties, codeveiligheid en gebruiksgemak door het gebruik van een overvloed aan technieken. Ten slotte biedt Rust uitgebreide ondersteuning voor parallelisme.

De implementatie is gebouwd op Apache DataFusion, wat een uitbreidbaar, open-source, in-memory, columnar data management systeem en query engine is. Apache DataFusion voorziet basisimplementaties van meerdere componenten die we nodig hebben om een zelfgemaakte Datalog engine te bouwen. Verder biedt het ook veel functionaliteit om logische plannen voor onze queries te construeren en te optimaliseren. Bovendien biedt Apache DataFusion andere prestatiegerichte functies en maakt het gebruik van het parallellisme dat wordt geboden door de Tokio bibliotheek van Rust.

Voor onze gegevens werken we met Parquet bestanden, die een kolomgericht binair formaat gebruiken. Het gebruik van Apache Parquet heeft verschillende voordelen: Parquet bestanden besparen ruimte door gebruik te maken van compressie, ze slaan datatypes en schema metadata op en ze zijn vrij efficiënt voor het lezen van grote datasets.

De zelfgemaakte implementatie verwacht verschillende invoerbestanden. De voorbewerking om de bestanden met Datalog programma's en relatietabellen voor te bereiden, gebeurt meestal met Python-scripts vanwege het gebruiksgemak van Python.

Voor de implementatielogica van de zelfgemaakte Datalog engine verwijzen we de gebruiker naar de volledige thesis omdat dit te diepgaand zou zijn voor deze samenvatting.

0.13 Vergelijking

Onze benchmarks zijn waar mogelijk uitgevoerd op vijf verschillende engines, namelijk: zelfgemaakte naïeve implementatie, zelfgemaakte semi-naïeve implementatie, zelfgemaakte semi-naïeve implementatie gecombineerd met magic sets rewriting, RecStep en Soufflé. Door onze eigen implementatie te vergelijken met RecStep en Soufflé krijgt onze vergelijking een bredere relevantie vanwege de verschillende ontwerpfilosofieën van deze Datalog engines.

Als benchmark Datalog programma's gebruiken we een subset van de programma's beschreven in de zesde sectie van de RecStep paper [42], samen met een variant van een van deze programma's, die kan worden gevonden in 'Modern Datalog Engines' [40] door Bas Ketsman en Paraschos Koutris. Deze programma's zijn afkomstig uit twee velden: grafiekanalyse en statische programma analyse. De benchmarks werden uitgevoerd met verschillende datasets. De specifieke datasets veranderden op basis van de karakteristieken van het huidige programma.

In onze benchmarks zien we veel patronen, sommige meer verwacht dan andere. Bijvoorbeeld, de overhead die gepaard gaat met andere technieken dan de naïeve evaluatie kan er voor zorgen dat ze langzamer zijn als de dataset te klein is. De uitvoeringstijden in deze gevallen zijn echter allemaal klein genoeg zodat dit geen probleem vormt. Als we meer kijken naar hoe de semi-naïeve evaluatie presteert, zien we dat onze zelfgemaakte implementatie iets beter lijkt te presteren dan RecStep voor kleinere datasets, terwijl RecStep iets beter lijkt te schalen. Soufflé daarentegen lijkt inconsistenter te zijn als het gaat om prestaties in vergelijking met de aangepaste semi-naïeve implementatie. Dit wordt toegeschreven aan het compilatiegerichte ontwerp, dat wisselende resultaten lijkt te geven op basis van het programma en de dataset.

Als we onze aandacht verleggen naar het evalueren van magic set resultaten, zien we verschillende niveaus van succes. Voor sommige benchmarks zorgt de magic set transformatie voor een significante verlaging in de uitvoeringstijd, terwijl voor andere benchmarks de magic set techniek er juist voor zorgt dat de evaluatie langer duurt dan een normale semi-naïeve evaluatie. Om deze metingen beter te begrijpen, hebben we ook het totale aantal afgeleide tupels tijdens de evaluatie onderzocht. Dit lijkt hetzelfde patroon te volgen als de uitvoeringstijd. Als de magic sets transformatie resulteert in een programma dat minder tupels afleidt dan normale semi-naïeve evaluatie, dan zal de uitvoeringstijd lager zijn dan bij semi-naïeve evaluatie. Als het resulterende programma daarentegen meer tupels oplevert dan normale semi-naïeve evaluatie, dan duurt het langer voordat de evaluatie klaar is.

We hadden gehoopt op een meer consistente en positieve invloed van de magic sets techniek, maar het resultaat is niet geheel verrassend. De efficiëntie van het programma dat het resultaat is van de magic sets transformatie, is afhankelijk van de gebruikte sideways information passing strategies. De zelfgemaakte implementatie van de magic sets transformatie is vrij eenvoudig en daarom is hier niet veel aandacht

aan besteed. Bijgevolg zouden de meetresultaten er heel anders uit kunnen zien in combinatie met een meer ingewikkelde implementatie van de magic sets transformatie. Een ander belangrijk aspect is de bindingsinformatie die door de query wordt geleverd. Als de set van bindingen kleiner is, dan zal de resulterende uitvoeringstijd ook kleiner zijn, omdat de bindingen selectiever zijn.

0.14 Conclusies

Uit onze benchmarkresultaten kunnen we concluderen dat semi-naïeve evaluatie-engines beter presteren dan naïeve evaluatie-engines voor elke datasetgrootte waarvoor optimalisatie interessant is en dat de magic set rewriting techniek sterk afhankelijk is van sideways information passing strategies en de grootte van de set van query bindingen.

Deze thesis heeft niet kunnen aantonen dat de magic set rewriting techniek in combinatie met semi-naïeve evaluatie definitief superieur is aan normale semi-naïeve evaluatie van een Datalog programma. Verder onderzoek naar dit onderwerp is noodzakelijk. Dit onderzoek kan verschillende onderwerpen verkennen, zoals: sideways information passing strategies en hun invloed, mogelijke integratie van de magic sets rewriting techniek in bestaande engines en het gedrag van de magic sets rewriting techniek in combinatie met complexere varianten van Datalog.

Contents

Chapter 1

Introduction

In our current data-driven world, being able to draw deep, meaningful insights from complex datasets is a crucial ability for everything from social media platforms and e-commerce recommendation systems to scientific research and cybersecurity. Many of these applications rely on being able to traverse relationships in data, such as finding connections in a graph or tracing dependencies in a system, which often require recursive reasoning.

As the volume and complexity of data continue to grow in almost every domain, the efficient evaluation of database queries becomes an increasingly vital and challenging problem in data management. This challenge is particularly important in fields such as data science, program analysis, declarative networking, and information extraction, where applications depend on large-scale relational data and complex query patterns. This often involves recursive queries, which are usually not easily expressed or efficiently processed by means of traditional SQL or naive query execution strategies.

Recursive queries, which allow a query to refer to its own result set, are a powerful tool for expressing certain computations such as transitive closure, hierarchical data traversal, and graph reachability. These types of queries are vital to various data-processing tasks. Examples of such tasks are: finding all dependencies in a software system, shortest path computation in graphs, et cetera. Efficiently executing recursive queries can pose a significant problem when it comes to computation and optimization. Traditional query evaluation techniques may fall short in both performance and scalability. This shortcoming scales with recursion depth and the size of the data.

In an effort to solve these challenges, many studies have been done into various optimization strategies, such as semi-naive evaluation, query rewrite rules such as the magic-set transformation, and so on. These techniques aim to increase efficiency by reducing redundant tuple derivations, limiting intermediate result sizes, and better utilizing the capabilities of underlying data processing engines.

A popular language for expressing recursive queries is Datalog, which stems from researchers realizing the limitations of SQL when expressing recursive logic, leading to its development. It is rooted in Prolog, but is designed specifically for use in database applications. Its rule-based formulation lends itself particularly well to expressing recursive queries in an intuitive and structured way. It originally got mainstream attention in the eighties and the early nineties and has recently seen a resurgence in both academic and commercial fields.

This thesis researches how specific query rewriting techniques and execution strategies can improve recursive query evaluation in Datalog-based systems, with a focus on scalability and runtime efficiency. We benchmark these techniques, using several datasets and Datalog programs, and compare how these techniques hold up against some state-of-the-art query engines.

The remainder of this thesis is structured as follows: Chapter two provides insight into how the Datalog language works, its semantics, and some of its restrictions and how to overcome them. As well as giving information about recursive query evaluation techniques. Chapter three investigates state-of-the-art Datalog query engines and how these engines are able to acquire their efficiency through optimization techniques, data structures, and more. Chapter four discusses the implementation and design choices for the custom Datalog engine utilized for acquiring empirical results. Chapter five takes a look at the benchmark results and the corresponding experiment setup. Finally, chapter six discusses possible

future work and provides a conclusion for this thesis, which talks about the findings and a personal reflection.

Chapter 2

Context

In this chapter, we explain Datalog. We cover the fundamental terminology of standard Datalog, the semantics of how we can interpret a Datalog program, the limitations of classic Datalog, and how to extend the language with extra semantics to overcome these limitations. On top of that, we also cover recursive query processing. Specifically, bottom-up evaluation, which derives rules from rules and input, and top-down evaluation, which attempts to answer a query by working backward from the goal. Lastly, we discuss how to combine the best of both bottom-up and top-down evaluation using the magic sets rewriting technique.

2.1 Datalog

Datalog is a declarative logic programming language. It's primarily used for querying databases and executing logical inference based on rules and the facts in the database. In more recent times, Datalog has started to gain popularity again due to a demand for high-level abstractions that support reasoning and rapid development in complicated, data-intensive systems. Datalog has a broad range of application domains, spanning both academic and commercial fields. One such example from the real world is DOOP (which we assume to be an abbreviation for Declarative Object-Oriented Program analysis or Datalog-based Object-Oriented Program analysis, though no documentation on the exact expansion of DOOP could be found). This is a pointer-analysis framework for Java, and it is basically a collection of program analyses expressed in the form of Datalog rules.

The structure of the following section mostly follows the flow and presentation of the second chapter found in Green's 'Datalog and recursive query processing' [1], as it provides a clear and systematic showcase of relevant concepts. As such, information and inspiration stem from this source as well. Thus, even though the contents of this chapter have been rephrased wherever possible, credit for the underlying material primarily goes to [1] and its sources.

2.1.1 Terminology

A Datalog program consists of a number of *rules* that define how new facts should be deduced from the facts already in the database. A rule consists of two primary components: the *head* and the *body*. The head is made up of a single predicate, where the predicate name represents the relation to which a new fact will be added, and the predicate arguments specify the structure of the deduced fact. The body of the rule is composed of a set of zero or more predicates that are combined through join operations to ensure the derived fact complies with the restrictions implied by the rule. If a rule has a body consisting of 0 predicates, it is called a *fact rule*. A fact rule is unconditionally true.

```
A(x, y) :- B(x, y), C(y, z).
```

Figure 2.1: Example of a Datalog rule.

The aforementioned arguments are typically called *terms*. These terms can be either constants or variables. A predicate, consisting of a predicate symbol and a list of terms as arguments, is typically called

an *atom*. In the specific case where every term of an atom is a constant, the atom is called a *ground atom*.

Predicates can be divided into 2 categories. These being *extensional database predicates* (EDB predicates) and *intensional database predicates* (IDB predicates). EDB predicates represent the source tables from the database, while IDB predicates represent tables containing the facts deduced by the Datalog program. Based on this distinction, we can also divide atoms into 2 categories: base atoms and derived atoms. These are atoms serving as entries in EDB predicates and IDB predicates, respectively.

A set of ground atoms is conventionally called a *database instance*. If this set contains only base atoms, it is called a *source database instance*. In other words, this is the database instance at the beginning of a Datalog program. The *active domain* of a database instance consists of the set of all constants that occur in the database instance.

Every rule in a Datalog program must satisfy the *range restriction property*. This property states that every variable in the head of a rule must abide by one of two following rules:

- The variable in question appears in an atom of the body.
  (e.g. $A(x)$:- $B(x)$.)
- The variable in question is equivalent to a variable that appears in an atom of the body.
  (e.g. $A(x)$:- $B(y)$, $x = y + 1$.)

For a fact rule to abide by the range restriction property, it follows that the head atom must be a ground atom.

For a Datalog program consisting of a set of rules P, the *Herbrand base* of P, $B(P)$, is the collection of ground atoms that can be formed from predicates and the constants in the active domain of P. A *groundinstance* of a rule can be acquired by substituting all variables of that rule with constants.

We provide some examples of the aforementioned concepts: Assume the Datalog rule from figure 2.1 makes up the entirety of a Datalog program P. Then B and C are EDB predicates because there are no rules deriving facts for these relations, and thus they will not change over the evaluation of the Datalog program. In contrast, A is an IDB predicate since the relation changes over the evaluation because of the rule deriving new facts.

Some examples of ground atoms for program P could be: $A(1, 4)$, $B(1, 2)$, or $C(2, 4)$. Using these ground atoms, we can also create an example of a ground instance of a rule:

$$A(1, 4) : -B(1, 2), C(2, 4).$$

Now, assume that the active domain of P is equal to the set $\{1, 2, 4\}$. Then the Herbrand base is:
$\{A(1, 1), A(1, 2), A(1, 4), A(2, 1), A(2, 2), A(2, 4), A(4, 1), A(4, 2), A(4, 4),$
$B(1, 1), B(1, 2), B(1, 4), B(2, 1), B(2, 2), B(2, 4), B(4, 1), B(4, 2), B(4, 4),$
$C(1, 1), C(1, 2), C(1, 4), C(2, 1), C(2, 2), C(2, 4), C(4, 1), C(4, 2), C(4, 4)\}$.

2.1.2 Semantics

The semantics of Datalog can be defined in three different ways. Although each way is very different from the other, in practice, they all yield the same result and are thus equivalent. The three semantics are:

- Model-theoretic Semantics
- Fixpoint-theoretic Semantics
- Proof-theoretic Semantics

In *model-theoretic semantics*, the focus is on what is computed. Datalog rules are seen as first-order logic constraints. So when interpreting the Datalog rule in figure 2.1 we get the following:

$$(\forall x \forall y \forall z)(B(x, y) \wedge C(y, z) \rightarrow A(x, z))$$

Given a source database instance I and a Datalog program P, a *model* of P is a database instance I' which extends I and satisfies all rules in P as constraints. There are many such models I' for a given

source database instance and program, but there is only one model that is smallest, called the *minimal model*.

Theorem 1. *For any Datalog program P and source database instance I, there is exactly one minimal model I' of P extending I. Thus I' is a submodel of any other model I'' of P ($I' \subseteq I''$). Furthermore, I' is polynomial in the size of I for a given program P.*

Proof. The theorem follows from the following two facts:

- For every Datalog program P and database instance I, there is at least one model I' attainable by adding all atoms over the active domain of I to instance I. Furthermore, I' is polynomial in the size of I, with the exact exponent being the maximal arity of any predicate in P.
- The intersection of two models is again a model.

□

Model-theoretic semantics states that the minimal model is the model that should be computed, and we denote it with $P(I)$.

In *fixpoint-theoretic semantics*, more attention is paid to how something is computed instead of to what is computed. This semantics is equivalent to the minimal model semantics but is more practical in application, as it is better suited to bottom-up evaluation strategies.

The fixpoint-theoretic semantics is based on the *immediate consequence operator* T_P for a program P. This operator maps database instances to other database instances. An atom A is an *immediate consequence* of a database instance I and a program P if either:

- A is a ground atom in I.
- A :- $B_1, B_2, ..., B_n$ is a ground instance of a rule in P and $B_1, B_2, ..., B_n$ are in I.

Accordingly, the immediate consequence operator T_P is defined as follows: $A \in T_P(I) \Leftrightarrow A$ is an immediate consequence of P and I.

Another notable aspect of the immediate consequence operator T_P is its *monotonicity*. This means that if database instance I is a subset of database instance J, then the immediate consequence of I will also be a subset of the immediate consequence of J.

$$I \subseteq J \Rightarrow T_P(I) \subseteq T_P(J)$$

A database instance for which $T_P(I) = I$ holds is called a *fixpoint*. Just like with the minimal models, there can be many fixpoints for T_P. When a fixpoint containing I is a subset of any other fixpoint for T_P containing I, then this fixpoint is called a *least fixpoint* for T_P. This least fixpoint contains no 'irrelevant' atoms and is equivalent to the minimal model $P(I)$ obtained in the model-theoretic semantics.

This semantics gives rise to a method for evaluating Datalog programs. The method relies on repeatedly applying the immediate consequence operator T_P to the source database instance I. Not only will this method eventually return a least fixpoint, but it can also be proven that this happens in a number of steps polynomial in the size of I. Since computation of T_P happens in polynomial time, it follows that the computation of a least fixpoint also happens in polynomial time.

An example is given in figure 2.2. Notice that $T_P(I)_3$ is equal to $T_P(I)_4$. We know the least fixpoint has been reached after the third application of the immediate consequence operator because there is no difference after applying the operator a fourth time.

In *proof-theoretic semantics*, we define the result of a Datalog program as the set of all atoms that can be proven using the source database instance and the rules of the Datalog program. This result is equivalent to the minimal model $P(I)$ for a source database instance I and Datalog program P.

If we want to prove atom $Tc(a, e)$ from the least fixpoint in figure 2.2, we can prove $Tc(a, b)$ by using a combination of the first rule in our Datalog program and the fact that $Arc(a, b)$ is given because it appears in the source database instance. Next we prove $Tc(a, c)$ using a combination of the second rule of our Datalog program, the fact that we've proven $Tc(a, b)$, and that $Arc(b, c)$ is given, because it occurs in the source database instance. Analogously, we attain $Tc(a, d)$ and then $Tc(a, e)$.

***Datalog program** P:*

- `Tc(x, y) :- Arc(x, y).`
- `Tc(x, y) :- Tc(x, y), Arc(y, z).`

***Source Database instance** I:*

Arc	
x	y
a	b
b	c
c	d
d	e

$T_P(I)_1$:

Arc	
x	y
a	b
b	c
c	d
d	e

Tc	
x	y
a	b
b	c
c	d
d	e
a	c
b	d
c	e

$T_P(I)_2$:

Arc	
x	y
a	b
b	c
c	d
d	e

Tc	
x	y
a	b
b	c
c	d
d	e
a	c
b	d
c	e
a	d
b	e

$T_P(I)_3$:

Arc	
x	y
a	b
b	c
c	d
d	e

Tc	
x	y
a	b
b	c
c	d
d	e
a	c
b	d
c	e
a	d
b	e
a	e

$T_P(I)_4$:

Arc	
x	y
a	b
b	c
c	d
d	e

Tc	
x	y
a	b
b	c
c	d
d	e
a	c
b	d
c	e
a	d
b	e
a	e

Figure 2.2: An example of the least fixpoint computation using a Datalog program for calculating the transitive closure on a simple source database.

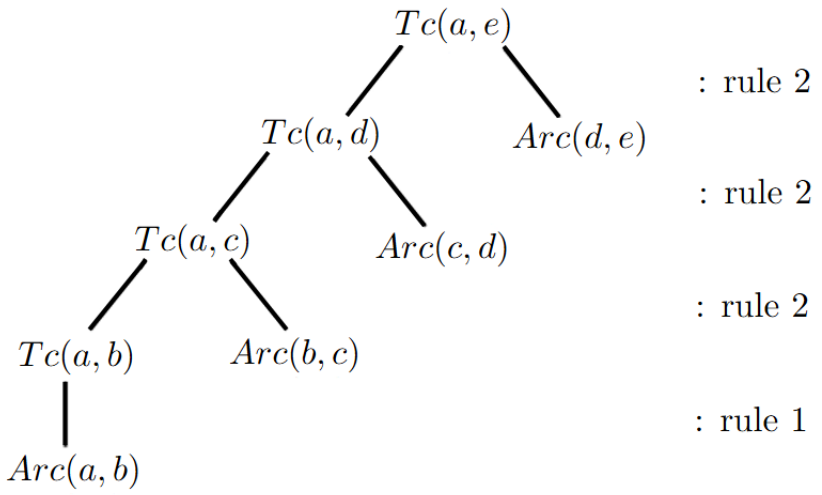

Figure 2.3: A proof tree for $Tc(a, e)$.

Such a proof can also be shown visually by means of a proof tree. The proof tree for the example above can be seen in figure 2.3.

Proofs of output atoms can be generated in two ways:

- Bottom-up or backward-chaining: deriving more and more proofs of atoms by applying rules repeatedly.
- Top-down or forward-chaining: trying to prove a candidate output by recursively trying to prove other atoms from which the goal atom can be proven.

Bottom-up is favored for use in Datalog, but top-down is also employed in practice, particularly in Prolog.

2.1.3 Negation

As mentioned earlier, Datalog programs are monotone: if $I \subseteq J$, then $P(I) \subseteq P(J)$. What follows is that core Datalog is incapable of expressing queries that are non-monotone. A logical solution to this problem is extending Datalog to allow for negation in rule bodies. This extension of Datalog is typically noted as Datalog$^\neg$.

A systematic treatment of Datalog$^\neg$ is more difficult than with normal Datalog, as not all theorems that held for Datalog also hold when extended with negation. Assume a program with the following rules:

- P :- not Q.
- Q :- not P.

No matter which semantics is used, a problem arises. This program now has two minimal models as opposed to the single unique minimal model we expect. These minimal models are $\{P\}$ and $\{Q\}$. These minimal models are also both least fixpoints, but neither is a unique least fixpoint. Even with proof-theoretic semantics, the meaning of the program is not clear. Neither P nor Q can be logically entailed from the rules, but a database instance containing neither P nor Q fails to satisfy the rules of the program.

A special semantics for Datalog$^\neg$ must be defined to avoid the problems caused by such programs. One of the simplest semantics is called semipositive Datalog$^\neg$. With this semantics, negation is restricted to EDB predicates in rule bodies. Furthermore, every variable that occurs in a negated predicate must also appear in a non-negated predicate. This safety condition is more commonly known as domain independence, and it guarantees that output is finite and depends only on the data in the database.

Since negation can exclusively be applied to EDB predicates in Semipositive Datalog$^\neg$, the negated predicates can be seen as EDB predicates in their own right. Predicate '$notP$' is then simply the complement of P over the active domain.

This allows us to define programs more freely than before. An example of one such program is the following, which computes if a node x can be reached from a node y through an indirect path:

- $Tc(x, y)$:- $Arc(x, y)$.
- $Tc(x, y)$:- $Tc(x, y)$, $Arc(y, z)$.
- $Indirect(x, y)$:- $Tc(x, y)$, not $Arc(x, y)$.

Model-theoretic semantics and fixpoint-theoretic semantics easily apply to semipositive Datalog$^\neg$ and we can state the following conditions (with P a semipositive Datalog$^\neg$ program and I being a source database instance):

- There is a unique minimal model J for P such that J agrees with I on the EDB predicates.
- There is a unique least fixpoint J for T_P such that J agrees with I on the EDB predicates.
- The minimal model and least fixpoint are identical and can be computed in polynomial time by applying T_P repeatedly to I.

In contrast to model-theoretic and fixpoint-theoretic semantics, proof-theoretic semantics is not easily applicable here. This is because the negation leads to needing to prove the non-existence of a proof subtree.

It is possible to extend the semantics for Datalog with negation even further to acquire even more inferencing power. The following semantics we look at is named stratified Datalog$^\neg$. We start with an example program that becomes possible with stratified Datalog$^\neg$:

- $Node(x)$:- $Arc(x, y)$.
- $Node(y)$:- $Arc(x, y)$.
- $Tc(x, y)$:- $Arc(x, y)$.
- $Tc(x, y)$:- $Tc(x, y)$, $Arc(y, z)$.
- $Unreachable(x, y)$:- $Node(x), Node(y)$, not $Tc(x, y)$.

In $Unreachable(x, y)$ we want all pairs of nodes where there is no path from x to y. The problem, however, is that if all rules fire immediately, then pairs of nodes will appear in $Unreachable$ that will also appear in Tc, but only after a few iterations. So what we actually want is for the first four rules to compute their results until nothing new is found and only then do we want the fifth rule to compute all pairs of nodes (x, y) without a path from x to y.

In the stratified semantics, a Datalog program P is actually a sequence of different semipositive Datalog$^\neg$ programs $P_1, P_2, ..., P_n$. When evaluating this sequence, the EDB predicates and the computed output for the IDB predicates of P_i will serve as the EDB predicates for P_{i+1}. We call this sequence of subprograms a stratification and define it as follows:
Definition 1. *A stratification of a Datalog$^\neg$ program P is an ordered partition of the IDB predicates in P into strata $P_1, P_2, ..., P_n$ such that both of the following conditions hold:*

- *Assume the following rule: A :- $..., B, ...$ of program P. If A is in stratum P_i and B is in stratum P_j, then $i \geq j$.*
- *Assume the following rule: A :- $...,$ not $B, ...$ of program P. If A is in stratum P_i and B is in stratum P_j, then $i > j$.*

A program is stratifiable if it permits the creation of a stratification. If we look back at the example program for computing $Unreachable$, then we see it is stratifiable because of the stratification with stratum 1 consisting of $Node$ and Tc, while stratum 2 consists of $Unreachable$. Thus separating rules 1 through 4 from rule 5.

For a stratifiable program P, the semantics are defined in two steps:

1. Stratify program P into subprograms $P_1, P_2, ..., P_n$.

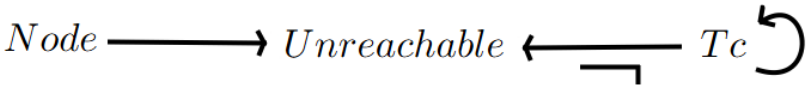

Figure 2.4: A finite precedence graph for the Datalog$^\neg$ program that computes *Unreachable*.

2. Evaluate the stratification in sequence, using the IDB predicates P_i as EDB predicates for P_{i+1}.

While a program P can have many stratifications, in actuality it does not matter which one is computed. This is because all stratifications yield the same result when applied to the same database instance. It follows that we can talk about the result $P(I)$ without ambiguity.

While we now know how to evaluate a stratified program, we still haven't mentioned how to stratify a program. This is done for a program P by constructing a finite precedence graph G_P with the vertices in this graph being the IDB predicates in P and the edges defined using the following rules:

- if A :- ..., B, ... is a rule in P, then (B, A) is an edge in G_P.
- if A :- ..., not B, ... is a rule in P, then (B, A) is an edge in G_P, but labeled with $\neg$.

Now we can check for stratifiability of a program P by ensuring its finite precedence graph G_P contains no loops including an edge labeled $\neg$. The graph can then be used to compute a stratification (in linear complexity to the number of IDB predicates in the program) by executing the following steps:

1. Compute the strongly connected components of G_P. These are the strata of the program.
2. Perform a topological sort to define an ordering for the strata.

An example of a finite precedence graph can be seen in figure 2.4. While this graph has a loop, this loop doesn't contain any edge with a $\neg$ label. The strongly connected components each contain one IDB predicate. After topological sorting, we learn that the component containing *Unreachable* must be evaluated last and that the order between the other two components is irrelevant.

2.1.4 Aggregation

Aggregation is a function present in almost all databases and database-evaluation systems. For example: counting how many times a product has been sold, or computing how many nodes are reachable from a certain node. Aggregate functions are mappings from bags of domain values to domain values. An aggregate term is an expression $f < t_1, t_2, ..., t_k >$ with f an aggregate function symbol (such as count, max, avg, ...) of arity k and $t_1, t_2, ..., t_k$ being non-aggregate terms. We allow aggregate terms to occur only in the head of a rule. Furthermore, a rule must satisfy an extended range restriction property:

- If a variable occurs in the head of a rule (including in aggregate terms), then the variable must also occur in the body.
- If a variable occurs in an aggregate term in a rule, then it cannot appear in the head outside of that aggregate as a non-aggregate term.

The variables in the head which appear outside aggregate terms are called the grouping variables. A ground instance of a rule with aggregates is obtained by replacing all grouping variables with constants.

Just like with negation, allowing recursive aggregation can cause semantic problems. For example, take a look at the following program:

- $P(x)$:- $Q(x)$.
- $P(sum(x))$:- $P(x)$.

When evaluating this, if the EDB predicate Q contains more than one fact, then in rule 2 an infinite loop of higher and higher sums will occur, assuming the database only works with positive numbers.

To avoid problems such as these, we once again rely on stratification. Using stratified aggregation, we avoid programs where aggregation occurs through recursion. We define stratification in the presence of aggregate terms as follows:
Definition 2. *A stratification of a Datalog¬ with aggregates program P is an ordered partition of the IDB predicates in P into strata $P_1, P_2, ..., P_n$ such that both of the following conditions hold:*

- *Assume the following rule: A :- $..., B, ...$ of program P. If A is in stratum P_i and B is in stratum P_j, then $i \geq j$.*
- *Assume the following rule: A :- ..., not $B, ...$ of program P. If A is in stratum P_i and B is in stratum P_j, then $i > j$.*
- *Assume the following rule: A :- $..., B, ...$ of program P. if A contains an aggregate term and is in stratum P_i and B is in stratum P_j and contains a variable occurring in the aggregate term, then $i > j$.*

A stratification for a Datalog¬ with aggregates program P can be found using an extension of the finite precedence graph. The process is analogous to the process for Datalog¬ programs with the addition of keeping track of aggregation. No loops containing an edge labeled with aggregation or negation should be present.

To define a fixpoint-theoretic semantics for programs with stratified aggregation, we extend the immediate consequence operator to handle aggregate functions. For a database instance I and a program P, assume a ground instance r of a rule in P with aggregates of the form: $R(t_1, ..., t_n)$:- $B_1, ..., B_n$.

Take all substitutions θ for all non-grouping variables of r such that $\theta B_1, ..., \theta B_n$ is in I. Group all θ into a set Θ. A fact $R(c_1, ..., c_n)$ is now an immediate consequence of I if the following conditions hold:

- The set of substitutions Θ is not empty.
- If t_i is a constant, then $c_i = t_i$.
- If t_i is an aggregate term $f(u_1, ..., u_k)$, then $c_i = f(\{(\theta u_1, ..., \theta u_k) | \theta \in \Theta\})$, with the curly braces implying a bag instead of a set.

We now define the immediate consequence operator T_P as follows: $A \in T_P \Leftrightarrow A$ is an immediate consequence of a ground instance of a rule in P for I.

Assuming aggregates are computable in polynomial time, we can reason that least fixpoint computation and consequently program evaluation are also computable in polynomial time. It is also possible to define a model-theoretic semantics for stratified Datalog¬ with aggregates, but we will not go into that here.

There are ways to optimize some programs that use aggregations. We briefly mention a technique called aggregate selection (more information in [7]). It boils down to applying a selection to certain predicates (filtering the predicate using an aggregate) to limit the facts passed to other rule bodies. This limits the number of facts derived during evaluation and can even lead to some program evaluations terminating where this would not be the case without aggregate selection.

2.1.5 Other Extensions

The extensions to Datalog we have mentioned thus far have all been computable in polynomial time. This is not the case by default, however.

One such extension, not computable in a polynomial timeframe, is the addition of arithmetic operators. The presence of this extension is no surprise, since many applications have a need for arithmetic. One might consider representing arithmetic equations as relations within the database. However, this approach encounters a fundamental limitation: it is infeasible to materialize every possible arithmetic computation as database tuples, as even the set of natural numbers is infinite—and consequently, so is the set of all arithmetic expressions involving them. Computing all calculations a program might need ahead of time is also not desirable. For this reason, a special extension is needed.

Another problem concerning mathematical calculations is the fact that they can cause non-termination when evaluating a Datalog program. For example, take a look at the rule head of the following rule:

$$Rank(i+1, v, sum(r)) : -Rank(i, u, s), Edge(u, v), EdgeCnt(u, c), r = 0.8 * s/c.$$

The $i+1$ ensures an infinite sequence of integers in the corresponding attribute. A lot of research [16,17] has been done in finding a semantics to properly handle the arithmetic, but often the engine executing the program evaluation leaves issues such as these to the user.

Another extension is the addition of functions to increase expressivity. Just like with the addition of arithmetic, useful properties (e.g. finiteness) of Datalog no longer apply with this extension present. The consequences of this are also often deferred to the user to solve.

As is to be expected, the list of extensions goes on. Some other notable extensions are listed below:

- The addition of a type system with primitive and complex types for relation attributes.
- The addition of a greedy choice operator, an operator that chooses a tuple non-deterministically during evaluation.
- etc.

2.1.6 Distributed Datalog

The last major extension of Datalog we will be exploring is for enabling distributed execution of Datalog programs. In this extension, distribution and exchange of data are explicitly captured in the program, while in a parallel execution of Datalog, these tasks are executed by the system.

A known example of a language that implements this extension is NDlog [46]. This variant of Datalog imposes restrictions that enable explicit management of data distribution and transfer. The programs in this language are intended to be evaluated on a physical network through the use of distribution. To account for network-related constraints, NDlog specifies addresses to indicate the position of data on the network. NDlog constrains the computations of the program to only happen over links provided to the program beforehand.

Another well-known variant of Datalog for distributed execution is WebdamLog [47]. In this variant, peers exchange rules. Every predicate has a special parameter to signify at which peer the predicate is located. This parameter can even be a variable bound by an earlier atom. Through this technique, data can be explicitly distributed and moved.

As a last example, we bring up Dedalus [48]. In this variant, every fact is parametrized with a location as well as a time. Using the time parameter, two categories of rules can be made: deductive rules, which work with facts from the same time, and inductive rules, which use facts from the previous time. The addition of a time parameter allows for clean minimal-model semantics. In actuality, there is a third category as well: asynchronous rules. In asynchronous rules, the relationship between time in head and body is unknown.

2.2 Recursive Query Processing

For every recursive query, there are multiple methods for evaluation. These can usually be classified as a bottom-up method or a top-down method. Bottom-up methods work by applying the rules of your program to all ground atoms, thus deriving new atoms in the form of rule heads that satisfy the rule bodies. The resulting minimal model is then materialized as a new database instance on which a select/project/join operation is executed to acquire the result of the query. Top-down methods work by pushing the selection criteria from the query, in the form of constants, down into the rules of the program necessary for computing the result of the query. This then creates new subqueries to be evaluated, which are handled similarly.

The structure of the following section mostly follows the flow and presentation of the third chapter found in Green's 'Datalog and recursive query processing' [1], as it provides a clear and systematic showcase of relevant concepts. As such, information and inspiration stem from this source as well. Thus, even though the contents of this chapter have been rephrased wherever possible, credit for the underlying material primarily goes to [1] and its sources.

2.2.1 Bottom-Up

A simple bottom-up evaluation algorithm is provided by the least fixpoint semantics. We evaluate in multiple iterations. Within each iteration, we evaluate every rule and derive new tuples using the immediate consequence operator T_P. We stop performing new iterations when we reach an iteration where no new tuples have been derived. This method is conventionally called the 'naïve method' and is probably what most people would do instinctively.

Let's look at the program from figure 2.2. One thing to notice when using the naïve method is that some tuples can be derived multiple times. This is not visible in figure 2.2, because duplicate tuples have been removed for the sake of readability. Yet it is easy to conclude that this does happen, since in every iteration the same rules are evaluated, while the Tc and Arc tables can only grow in size (or stay the same size when no new tuples are derived).

This problem is addressed by the semi-naïve method, which aims to minimize the number of redundant derivations. This method relies on the idea that no computation from the previous iterations should be repeated. This is realized through the concept that only the new tuples from the last iteration can lead to new derivations. These new tuples are called 'deltas'. Additionally, we want to efficiently compute the deltas of the current generation for use in the next generation. The set of tuples for a predicate for any iteration i is equal to the union of the tuples for that predicate from iteration $i-1$ and the tuples in the computed deltas from iteration i.

For explanation purposes, assume a Datalog rule with head P that is *mutually recursive* with IDB predicates $P_1, ..., P_n$ (mutually recursive means there must also be rules with $P_1, ..., P_n$ where P is in the body of those rules) and in which the EDB predicates $Q_1, ..., Q_m$ also appear.

$$P :- P_1, ..., P_n, Q_1, ..., Q_m.$$

We refer to the set of tuples of a predicate at a given iteration by adding a superscript with the iteration in question (e.g.: $P_1^{[i]}$). We refer to the set of tuples derived in an iteration i by using the following notation: $\delta(P)^{[i]}$. With this new notation, we can write down the formula for the tuples of a given iteration:

$$P^{[i+1]} = P^{[i]} \cup \delta(P)^{[i]}$$

What follows now is that we can also create a more detailed rule for computing $P^{[i]}$ for every $i > 0$ by applying the equation above to the original rule:

$$P^{[i+1]} :- P_1^{[i]}, ..., P_n^{[i]}, Q_1, ..., Q_m.$$
$$\Leftrightarrow P^{[i+1]} :- (P_1^{[i-1]} \cup \delta(P_1)^{[i-1]}), ..., (P_n^{[i-1]} \cup \delta(P_n)^{[i-1]}), Q_1, ..., Q_m.$$

We should note that the set of derived tuples is not actually exclusively new tuples. The set of exclusively new tuples (the deltas) for a predicate P is actually computed by calculating the difference of the set of derived tuples $\Delta(p)^{[i]}$ (also called the true deltas) and the tuples from the predicate $P^{[i]}$:

$$\delta(P)^{[i]} = \Delta(P)^{[i]} - P^{[i]}$$

The pseudocode for the semi-naïve method can be seen in algorithm 1 [1]. Each repeat-until loop is one iteration where evaluate runs all the delta rules below, with $\Delta(p)^{[i]}$ being equivalent to the union of all the rules with $\Delta(p)^{[i]}$ as the head (for a proof of correctness, refer to [8]):

$$P^{[i+1]} = P^{[i]} \cup \Delta(P)^{[i]}$$

$$\Delta(P)^{[i]} :- \delta(P_1)^{[i-1]}, P_2^{[i-1]}, ..., P_n^{[i-1]}, Q_1, ..., Q_m.$$
$$\Delta(P)^{[i]} :- P_1^{[i]}, \delta(P_2)^{[i-1]}, P_3^{[i-1]}, ..., P_n^{[i-1]}, Q_1, ..., Q_m.$$
$$\Delta(P)^{[i]} :- P_1^{[i]}, ..., \delta(P_n)^{[i-1]}, Q_1, ..., Q_m.$$

Algorithm 1 Semi-Naïve evaluation

for each *IDB predicate* p **do**
  $p^{[0]} := \varnothing$
  $\delta(p)^{[0]} :=$ *tuples produced by rules using only EDB's*
end for
$i := 1$
while ($\delta(p)^{[i]} \neq \varnothing$ for any IDB predicate p) or $i = 1$ **do**
  $p^{[i]} := p^{[i-1]} \bigcup \delta(p)^{[i-1]}$
  evaluate $\Delta(p)^{[i]}$
  $\delta(p)^{[i]} := \Delta(p)^{[i]} - p^{[i]}$
  $i := i + 1$
end while

When executing both the naïve method and the semi-naïve method on the program from figure 2.2, we can confirm that both methods yield the same result. However, if we look at the number of times certain tuples get derived, we see a decrease in the number of derivations for many tuples.

An example of this difference in derived tuples can be seen in figure 2.5.

When using the semi-naïve method on programs with negated predicates, we do not need to take extra precautions. This is because of the stratification allowing us to treat negated predicates the same as EDB predicates since they must be computed in a lower stratum.

2.2.2 Top-Down

Top-down evaluation also focuses on limiting redundant derivations, but instead of focusing on the redundancy over iterations, top-down evaluation focuses on avoiding derivations that are unnecessary for the query. When using top-down, you start from the query and push down the selection criteria from the query into the rules. It can be seen as the search for a proof tree from the proof-theoretic semantics, since the derived tuples are exactly those that appear in the proof tree.

The most popular technique for top-down evaluation is Query-Subquery (QSQ). To provide a clear explanation of this technique, it is necessary to first introduce and define some key terminology.

The first thing to introduce is *adorned* predicates. These are predicates with some arguments bound. This boundedness signifies if an argument should receive binding information. In other words, if an argument is bound, then information is provided to specify how the argument should be replaced by a constant during the construction of a subquery. For a predicate P, we refer to the adorned predicate as P^γ. γ is a sequence of b's and f's. The first character of γ is b if the first argument of P is bounded; otherwise, it will be an f. This pattern extends analogously to the second argument and the second character of γ, and so on for subsequent arguments. It follows that the length of γ is equal to the number of arguments in P.

The binding information we pass to the adorned predicates is called the binding relations and is conventionally represented using R. R is the set of all bindings for the bounded arguments in P^γ. For example: $R = \{\{x \mapsto 3, z \mapsto 9\}, \{x \mapsto 5, z \mapsto 9\}\}$. The adorned predicates and binding information together $\langle P^\gamma, R\rangle$ represent the subqueries to be answered. These subqueries are constructed by changing the bound arguments from P^γ with constants as defined in the binding information in R.

We rewrite rules that derive P into adorned rules that derive P^γ, so that we can use these new rules to determine how the subqueries are constructed. To acquire these adorned rules, we must replace every predicate in the original rule body by an adorned predicate. Adornment is determined as follows: an argument is bounded if any of the following conditions hold:

- If the argument is a constant.
- If the argument is a variable which has already been bound in the rule head through unification of the rule head with binding information from the query.
- If the argument is bound by some variable in an atom left to it in the rule body.

***Datalog program** P:*

- `Tc(x, y) :- Arc(x, y).`
- `Tc(x, y) :- Tc(x, y), Arc(y, z).`

***Source Database instance** I:*

Arc	
X	Y
a	b
a	d
b	b
b	c
c	d

***Derived tuples** Tc:*

***Iteration** 1:*

Tc			
naïve		semi-naïve	
X	Y	X	Y
a	b	a	b
a	d	a	d
b	b	b	b
b	c	b	c
c	d	c	d

***Iteration** 2:*

Tc			
naïve		semi-naïve	
X	Y	X	Y
a	b	a	c
a	d	b	c
b	b	b	d
b	c		
c	d		
a	c		
b	d		

***Iteration** 3:*

Tc			
naïve		semi-naïve	
X	Y	X	Y
a	b	a	d
a	d		
b	b		
b	c		
c	d		
a	c		
b	d		

Figure 2.5: The tuples derived for each iteration during the naïve and semi-naïve (a.k.a. the true deltas) evaluation of a Datalog program for computing transitive closures.

We provide a few examples to clarify. Assume a query q that specifies we want to find the results of a predicate A where the first argument is bound by some values and we have a Datalog program consisting of the following rule:

$$A^{bf}(x, y) : -B(1, z), C(y, x, z).$$

We have already determined the adornment of the head through the binding information in the provided query. Next, we look at the first predicate in the rule body. The first argument is a constant and is thus bound according to our first condition for adornment. The second argument is not a constant and neither appears in the rule head nor in a predicate of the rule body to the left of the current predicate. Thus, we can conclude the second argument is free. We now obtain the following adornment for our first predicate of the rule body.

$$A^{bf}(x, y) : -B^{bf}(1, z), C(y, x, z).$$

Now we look to the last predicate in the rule body. The first argument y does appear in the head of the rule, but as it is not bound in the head, it stays free in the last predicate. Argument x also appears in the head of the rule, but in contrast to y, x is bound in the head and is thus also bound in this predicate. Lastly, z appears to the left of this predicate in the rule body. Although z is not bound in the previous predicate, it is actually bound here because of its earlier appearance. The binding information for z is derived from finding all values for z that satisfy $B(1, z)$. All this information allows us to determine the adornment of the last predicate in the rule:

$$A^{bf}(x, y) : -B^{bf}(1, z), C^{fbb}(y, x, z).$$

The binding relations can be categorized into two groups: binding information passed top-down through unification of the queries and the head of the rules, and binding information passed sideways from atom to atom in the rule body. Passing top-down binding relations is done using 'input relations'. An input relation $input\_P^{\gamma}(V)$ denotes binding relations passed down into the head of rules deriving P^{γ} with V being a set of bounded arguments. Consequently, $\langle P^{\gamma}, input\_P^{\gamma} \rangle$ represents the subqueries obtained through unification. For passing sideways binding information, we use 'supplementary relations'. $sup_j^i(V)$ is the supplementary relation that denotes the binding information passed into the j^{th} atom in the i^{th}

adorned rule with V being a set of bounded arguments by the atoms in the same rule body before the j^{th} atom and which appear in the head atom or a body atom with an index larger or equal to j. A special case of the supplementary rule is $sup_n^i(V)$ with n being the number of atoms in the rule body. This is the supplementary relation containing the binding information acquired after evaluating all the subqueries constructed by the rule body. In other words, $sup_n^i(V)$ contains the answer to the head of the query.

Now a description of the algorithm for Query-Subquery:

1. Unify the query atoms with the rules to acquire adorned rule heads. Use the information from the adorned rule heads to determine the adornment of the corresponding rule bodies. For each IDB predicate the query has unified with, an input relation is created and populated with the bindings provided by the query.
2. Create the supplementary relationship sup_0^i by projecting out the necessary variables from the input relations for every i ranging from 0 to the number of rules.
3. Producing and evaluating subqueries, and computing subsequent supplementary relations happen interleaved.
   (a) Assume $P^\gamma(V)$ to be the j^{th} atom in the rule body. We construct subqueries from $\langle P^\gamma, sup_j^i(V')\rangle$ by substituting variables in $P^\gamma(V)$ with constants from $sup_j^i(V')$.
   (b) A new supplementary relation $sup_{j+1}^i(V)$ is constructed by executing a join between $sup_j^i(V')$ and the j^{th} atom.
   (c) Pass the binding information from $sup_j^i(V')$ to $input\_P^\gamma$ so that the subqueries can be evaluated using the same evaluations steps.
4. The final supplementary relation is used to compute the answer set for the head of the rule.

QSQ has an iterative implementation where you iterate over these 4 steps until no more subqueries are created and no more tuples are added to any answer set. There is also a recursive implementation. For a deeper dive into these implementations, refer to [9].

2.2.3 Magic Sets

Top-down evaluation methods, such as Query-SubQuery, reduce redundancy by deriving only those facts necessary for answering the query. This is done by pushing binding information from the query into the evaluation of the rules. Bottom-up evaluation methods, such as the semi-naïve method, reduce redundancy by limiting the number of times the same tuple gets derived during evaluation. Ideally, we combine both methods to further reduce redundancy. Much research has already been done in this field to let bottom-up evaluation methods make use of the information in queries. One technique that has become the subject of study is 'magic sets'. Magic sets is a family of rewrite techniques that for a Datalog program P produces another Datalog program P' such that, for any database instance I and query q, the answer computed for P is equal to the answer computed for P'. Moreover, the facts derived by a semi-naïve evaluation of P' are exactly those that would be derived by a QSQ evaluation of P. The idea behind magic sets is that the binding information used in QSQ can be portrayed as predicates, which can then be introduced into rule bodies. The resulting joins including those predicates will hopefully constrain the facts derived by bottom-up evaluation of the rules. There is a strong link between magic sets and QSQ. This is because the magic sets transformation can be viewed as a method for incorporating the supplementary relations, introduced by QSQ, directly into the rules of the program P. These supplementary relations will have to be constructed through the use of bottom-up evaluated Datalog rules and by using these relations in rule bodies for sideways information passing, we avoid top-down information passing.

There are three broad steps for a magic sets rewrite:

1. Infer the adorned rules from the original rules, with the adornments determined by the query.
2. Define the binding predicates and determine which rules derive into them.
3. Add the binding predicates into the adorned rules.

$$Tc(x,y) :- Arc(x,y).$$
$$Tc(x,y) :- Tc(x,y), Arc(y,z).$$

$$Sup_0^1(x) :- Input\_Tc(x).$$
$$Tc^{bf}(x,y) :- Sup_0^1(x), Arc^{bf}(x,y).$$
$$Sup_0^2(x) :- input\_Tc(x).$$
$$Sup_1^2(x,y) :- Sup_0^2(x), Arc(x,y).$$
$$Input\_Tc(y) :- Sup_1^2(x,y).$$
$$Tc^{bf}(x,z) :- Sup_0^2(x), Arc^{bf}(x,y), Sup_1^2(x,y), Tc^{bf}(y,z).$$

Figure 2.6: A Datalog program for computing transitive closures and its rewritten version using magic sets rewrite for a query binding the first variable of relationship Tc. The adornment is left in for clarity but means nothing as the actual binding information is passed through the supplementary relations and the input relation.

How to infer the adorned rules has already been mentioned earlier, so the explanation for this step will be passed over. The binding predicates consist of two groups: the input relations and the supplementary relations. The input relations begin with constants determined by the query bindings. The rules for the supplementary relations are defined next. The 0^{th} supplementary relation for a rule is computed by projecting out the necessary variables from the input relation. Any other supplementary relations are computed by joining the atom to its left with its corresponding supplementary relation. For the final step, we take the original rules and add in the binding predicates for which we just computed the rules. The 0^{th} supplementary relation comes before the first atom of the original adorned rule, the 1^{st} supplementary relation comes before the second atom of the original adorned rule, and so on. Do not forget to properly update the input relation to correctly constrain the j^{th} atom. Do this by adding a rule that updates the input relation with $sup_j^i(V)$. Furthermore, notice that we skip the computation of supplementary relation $sup_n^i(V)$, with n being the number of predicates in the rule body. The creation of this supplementary relation can be omitted because we can simply take the results of the rule body that would be used in the rule for this supplementary relation and add these tuples directly to the relationship of the original rule.

The input and supplementary predicates are sometimes also referred to as magic predicates.

For an example of what the result of a magic sets rewrite looks like, refer to figure 2.6. Note that $Input\_Tc$ starts with tuples present already.

There are a number of optimizations you can make while using the magic sets rewrite. For example, it is possible to limit every rule to a maximum of two atoms. This is because every supplementary relation is already a join of the atom before it and the previous supplementary relation. For this reason, you can leave out all the atoms before the last supplementary relation in the rule body. In the magic sets rewrite in figure 2.6, this would change the last rule to $Tc(x,z)$:- $Sup_1^2(x,y), Tc(y,z)$.

Another possible optimization is to change the order of the atoms in the rule bodies of the original program. This will result in a different program after the magic sets rewrite.

These 'optimizations' are called sideways information passing strategies, also abbreviated as SIPS. It is important to double-check the effectiveness of the chosen strategies, as these can even have a counter-productive effect on the efficiency of your evaluation if chosen poorly because of the extra computation of binding predicates and the extra execution of joins in rule bodies. The most effective SIPS are those

that maximize the selectivity of the magic predicates while minimizing computation costs.

There are three important factors that influence the effectiveness of a choice for a SIPS:

- Establishing an appropriate ordering of atoms in rules to constrain expensive computations.
- Choosing atoms for the computation of binding predicates to guarantee that their selectivity is worth the computation cost they require.
- Determining which variables should be bound within each atom (what tuples are stored in the binding predicates) in order to minimize the size and consequently the cost for materializing these predicates.

When it comes to determining the optimal ordering of atoms in rules, there is a strong link to choosing a join ordering for query evaluation. This link has already been explored by multiple other studies: [10], [11], and [12]. There are two representative approaches for this problem: a runtime approach and a static approach.

In the runtime approach, a relational operator is introduced for computing a binding predicate, and then introducing it in the body of a query. The ordering of the atoms is optimized here for the computation of binding predicates for a magic sets rewrite. This operator is called a 'filter-join'. When the filter-join is executed on a relation R and a relation S, then the join attributes of R are projected out into a separate 'filter-relation'. Only the tuples of S that join with the filter-relation are then joined with R. This is how we mimic the creation of a binding predicate. We rely on the query optimizer to add this operator only when it has a positive effect on the overall cost of computation. The query plan will then specify what atoms to incorporate in the creation of the binding predicate. As of yet, this approach has only been implemented for non-recursive queries.

In the static approach, approximate statistics on the contents of the predicates are of importance. This does not just limit itself to EDB predicates either. Statistics on IDB predicates are computed using abstract interpretation. Using these statistics, heuristics are employed (similar to those employed by the query optimizer in the runtime approach) to determine an optimal atom order. An important asset for this approach is the predicate dependency graph, which is a static representation of predicate statistics. For a certain predicate $P(x_1, x_2, ..., x_n)$, a predicate dependency graph is a triple $(\sum, G, \prod)$ consisting of the following parts:

- $\sum$: Function for assigning an estimated size for each argument x_i.
- G: Set of arrows between arguments ($x_i \xrightarrow{\alpha} x_j$), indicating that for every x_i there are α x_j's. In other words, the size of the set of values for x_j is approximately α times the size of the set of values for x_i.
- $\prod$: Set of equality constraints of the form $x_i = x_j$.

An example of a predicate dependency graph can be seen in figure 2.7.

The predicate dependency graph for EDB predicates is based on pure statistics, but as mentioned before, for IDB predicates, some of these statistics are approximated using an abstract interpreter. This abstract interpreter uses abstract values (pessimistic upper bounds) of predicates from the rule body to calculate statistics for the rule head. Every relational operator has its own abstract interpretation. We give the example of the intersection $A \wedge B$ here, but interpretations for other relational operators can be found in [11].

The three aspects of the predicate dependency graph for $A \wedge B$ are determined as follows:

- $\sum(x) = min(\sum_A(x), \sum_B(x))$ for any column x. This is because in an intersection, the result cannot be greater than either column.
- If $\alpha_1 = \alpha_2$ for $x \xrightarrow{\alpha_1} y \in G_1$ and $x \xrightarrow{\alpha_2} y \in G_2$, then the result is trivially determined to be $x \xrightarrow{\alpha} y \in G$. However, if this is not the case, then $\alpha = min(\alpha_1, \alpha_2)$, because we can pick the best, or smaller, since both α_1 and α_2 are pessimistic approximations.
- all equalities hold after $\prod = \prod_1 \cup \prod_2$.

If recursion is present in the rule body, then the recursive computation of the predicate dependency graph is executed a fixed number of times. Three times seemed to give a satisfactory result, according to [11].

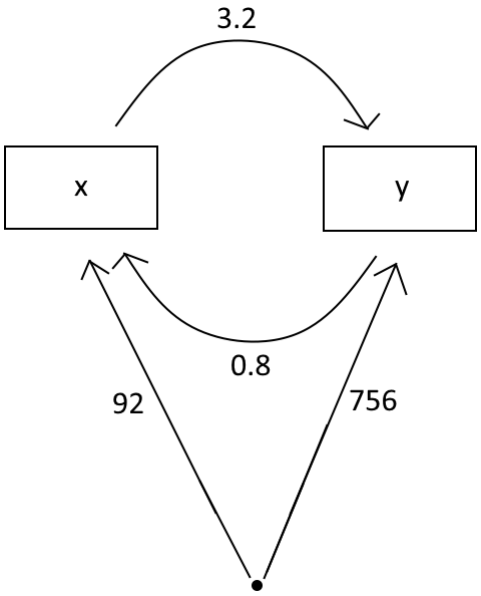

Figure 2.7: An example of a predicate dependency graph for a predicate with arguments x and y.

The estimated size of a predicate can now be computed by calculating the weight of the minimum spanning tree with edge weights computed using multiplication instead of addition. If we look back at figure 2.7, we can see there are two options for the size of the predicate here ($92 * 3.2$ or $756 * 0.8$). This is because of the estimated nature of this technique. We can choose the lesser of both options since these are already pessimistic upper bounds.

Now that predicate sizes are available, a greedy heuristic is employed to order the atoms with the smallest atom going first. Predicate sizes must always be computed in context. It follows that the size of a predicate can differ if we take the abstract values of already ordered atoms into account. The estimated size of a predicate A in the context of another predicate B is the conjunction of the formula for both predicates $A \wedge B$.

Now we know how to order atoms. Another important factor is how to define the used binding predicates. The challenge lies in reducing cost while enhancing selectivity. A binding predicate can be constructed from any combination of predicates that appear to its left in the rule body. An instinctive way to optimize the definition of a binding predicate is to look at the selectivity of the predicates used to create it. If one of the predicates does not enhance selectivity a lot, it might be a good idea to leave it out.

How do we check the selectivity in practice? We can make use of the same assets as we used for determining the atom order. In the runtime approach, the filter-join gives us information on what to include in our binding predicate. R filter-join S implies that the atoms used in the creation of R should be used to create our binding predicate. In the static approach, the key lies in the estimated size of the conjunction of two predicates. For example: if the estimated size of $A \wedge B$ is equal to or close to the estimated size of A, then the selectivity of B might not be worth the cost.

A last thing to consider is which variables should be bound. Just like a predicate does not always provide enough selectivity for the cost it brings, so does not every binding of a variable bring an adequate amount of selectivity. For example, assume the following program:

$$P(x, y) :- x = 1, type(y).$$

$$Q(x, y) :- x = y.$$

$$Query(x, y) :- P(x, y), Q(x, y).$$

In the third rule, y is only being constrained by the binding of x to a constant. Binding y would not add any more selectivity.

We briefly mention here that research has been done to extend the magic sets technique to include other useful information, such as arithmetic comparison constraints among other information. More info on this and other extensions can be found in [13], [14], and [15].

Chapter 3

An overview of Datalog Systems

Datalog has been used in a large range of fields. Over time, various engines have been developed, sometimes tailored with a specific application domain in mind. While each shares foundational principles, they differ from one another in notable ways. Some of the more well-known (and more recent) engines include:

- Graspan [24]
- Soufflé [22]
- BigDatalog [19]
- RecStep [23]
- LogicBlox [18]
- SociaLite [20]
- bddbddb [21]

Yet there are many more.

In the following section, we discuss key features of Datalog engines such as optimization techniques, data structures, et cetera, and analyze how their implementation differs among the various systems. A lot of inspiration and information for this section were drawn from [40]. Thus, even though the contents of this chapter have been rephrased wherever possible, credit for the underlying material primarily goes to [40] and its sources.

3.1 Query Plan Generation

Assume a rule from a Datalog program. To compute this rule, a query q must be executed. This query consists of the union of a number of join-project queries (joining the predicates and then joining this intermediate result with the tuples from the predicate from the rule head). While the query q for this rule remains unchanged throughout the evaluation of the Datalog program, the corresponding query plan might not necessarily stay the same. This is because the size of the IDB predicates changes over the iterations.

There are two ways to decide a query plan for q. One can choose a dynamic or a static query plan. When using a dynamic query plan, the system will attempt to find an optimal plan for the query during each iteration. On the other hand, if one chooses a static query plan, then the query plan is determined once ahead of time and then used in each iteration. Both methods have their advantages and disadvantages.

Static plans can degrade over a while. For example, take a rule containing an EDB predicate and a predicate for the deltas of a relation. If a hash-join is computed, then we might want the hash table to be constructed on the side of the EDB table in the earlier iterations since there will still be a large number of deltas, while in the later iterations, when there are probably significantly fewer deltas, we might want the hash table to be constructed on the table containing the deltas.

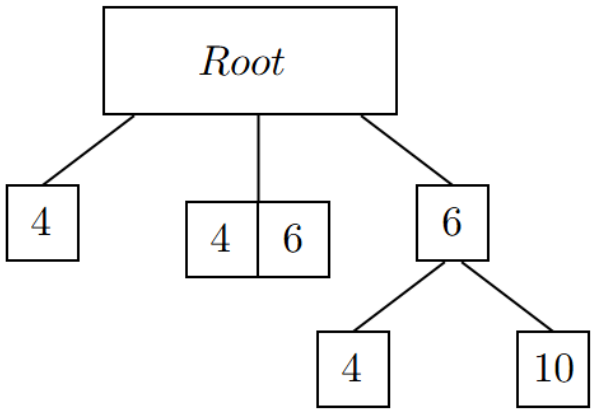

Figure 3.1: An example of a versioned B-tree. In the first iteration, 4 was added. Next iteration, 6 was added. In the final iteration 10 was added. Each version of the B-tree is represented by a branch from the root. Here, the first version is the left-most branch, the second version the middle branch, and the last version is the right-most branch.

Dynamic plans require the collection of statistics on IDB relations every iteration. This can be a costly operation. It is possible to relieve the burden of this operation on execution time slightly by collecting lightweight statistics with the operators we want to use in mind. This improvement is employed by RecStep, which is an interpreter. However, this method is more complex to incorporate into systems that compile Datalog because of the need to use runtime information.

Besides the choice for static or dynamic query plans, there is also the aspect of plan optimization. RecStep does this by bundling the rules with the same head into a single query and then passing that query to its underlying relational database management system's query optimizer called QuickStep. Unlike RecStep, Soufflé uses a static plan for every rule and on top of that, it always uses a nested loop join. However, it optimizes the nested loop join using specialized concurrent data structures (B-trees, tries).

Plan optimization is rather complex. This is caused by the fact that statistics for later iterations do not exist and are difficult to estimate. For this reason, a non-negligible number of engines choose a simple plan. For example, BigDatalog simply executes joins between the predicates of a rule body from left to right. Some engines, such as Myria and Naiad, even offload the task of providing a query plan to the user.

3.2 Difference Computation

The difference operator plays an important role in semi-naïve evaluation, as it is intensively used to compute the various delta relations. To avoid this operator becoming too much of a bottleneck, it must be highly optimized. Some engines use specialized data structures for this. Soufflé uses a B-tree for this, which is optimized for this purpose. RecStep employs a specially tailored dynamic set difference operator, which changes the algorithm used depending on the size of the IDB predicate compared to the size of the set of new facts. An alternate, effective data structure for this task is the versioned B-tree, which is a tree where every child of the root represents a version of the B-tree. This data structure works efficiently because versioned B-trees share the previous iterations as a common history. A simple example can be observed in figure 3.1.

3.3 Parallel Execution

The ability to process data in parallel has grown in importance over time because of the growing sizes of datasets. Unfortunately, not all Datalog programs are easily parallelizable. Whether a program permits an efficient parallelization depends on if the program is in complexity class NC, and there do not seem to be algorithms for deciding this as of yet.

3.3.1 Shared-Nothing Datalog Engines

Shared-nothing architectures are architectures that rely on a large network of independent processors/machines connected via a fast communication network. A significant number of engines use this architecture, such as:

- BigDatalog
- SociaLite
- ...

Parallel Datalog engines utilize many of the same techniques used in classic parallel query processing. This is because semi-naive evaluation is, simply put, just the execution of a sequence of relational queries. All relations are partitioned to the machines using partitioning methods. Based on the partitioning scheme, the Datalog engine can choose a parallel query plan for every rule, and it can form communication patterns to guide the newly produced facts to the right machines.

Naturally, the specifics of the implementation differ from engine to engine. One such aspect in which the engines may differ is the choice of data partitioning strategy. EDB relations only have to be partitioned once, since they do not change. IDB relations, on the other hand, may need to be repartitioned every iteration. This is because a parallel join operator partitions each relation by its join key, but new tuples might end up in different blocks of the partition. Consequently, optimizing this process and thus the data communication is an important task for the engine.

For the more complex strategies, we refer back to section 2.1.6. However, other parallel Datalog engines employ simpler partitioning methods. The standard partitioning method is hash partitioning, which is a technique popular in relational database management systems. Engines that use hash partitioning include: BigDatalog, SociaLite, Myria, among others. When using this technique, each relation will determine a subset of its attributes to be partitioned by. However, the input partitioning of an operator and the chosen partitioning may not match, and then an extra reshuffling step must be present. The choice of attribute subset differs per engine. In BigDatalog, this is always the first attribute. In SociaLite, the user can decide on a subset. SociaLite also employs a second partitioning method, known as range partitioning. This method works by dividing a range into subranges, which are then spread evenly over the machines. SociaLite makes the user specify the communication pattern according to the chosen partitioning method. This user specification limits scalability to larger programs.

Certain Datalog programs permit a highly efficient parallel evaluation in which no more IDB relations need to be communicated between machines after the original broadcast of EDB facts and wherein no redundancy is present. This results in no more need for synchronization during the computation, which can save some overhead costs. However, a major disadvantage lies in the original broadcast, which can be costly if the EDB relations are on the larger side. These kinds of programs are called decomposable programs. The program for computing transitive closures in figure 2.2 is an example of such a decomposable program. If we hash-partition based on the first attribute, communication between machines will not be necessary until after computation has finished (to gather the result on one machine). Every tuple derived through the recursive rule will already be on the correct machine. An example can be seen in figure 3.2. When comparing the right-most table with the minimal model for the transitive closure program, we can see that all facts starting with a have indeed been derived in the partition.

Sadly, there is no way to systematically determine the decomposability of a program. However, several sufficient conditions for the decomposability of single-rule programs have been identified, which are based on the existence of a pivot [25]. For the recursive predicates in the single-rule program, find a subset of argument positions. If for this subset the following conditions hold, then any argument position from this subset is a pivot.

- For all recursive predicates and the rule head, the same variables at the argument positions (with the order being a non-issue).
- The variables appear the same number of times in each recursive predicate and in the rule head.

As an example, assume the following recursive single-rule program:

$$A(x, y, z, z) : -A(v, z, z, y), A(w, z, y, z), B(x, v, w).$$

For this program, arguments 2, 3, and 4 are pivots, thus the program is decomposable.

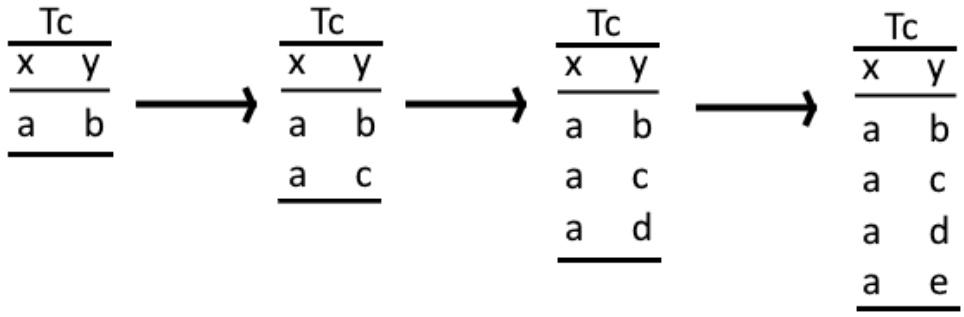

Figure 3.2: Execution of the recursive second rule of the program for computing transitive closures, which can be seen in figure 2.2. This partition is for computing all facts of the Tc relation that have a as a first attribute. The left-most table of Tc is after the execution of the first rule for the partition.

The concept of a pivot was later extended to full Datalog through the introduction of generalized pivoting. This technique is used in BigDatalog. If a pivot set is found, then decomposability is confirmed and an appropriate partitioning is found.

Another aspect critical to parallel evaluation is whether different processors work synchronously or asynchronously. If the execution is synchronous, then after every iteration all machines have to wait until every other machine has finished their computations. Semi-naïve evaluation lends itself well to synchronous execution because of the presence of iterations where machines need access to facts from a previous iteration. However, synchronous execution introduces an obvious candidate for computational bottlenecks since all machines must wait for the slowest member. In Datalog especially, one must take care because the number of iterations (and thus the number of synchronization points) can quickly add up.

Asynchronous execution does not suffer from synchronization bottlenecks and will not suffer from a lack of load balancing. However, asynchronous execution has its own set of issues. First of all, not all programs lend themselves to asynchronous evaluation. For example, think of negation and aggregation, which are dependent on the completion of the computation of another relation. Second of all, not all parallel systems support asynchronous execution of their operators, as this requires the ability to pipeline data between operators without the need for synchronization. Third of all, asynchronous evaluation can lead to redundancy of computations, as it might access stale data from other machines. Underneath is a list of some engines and the methods they chose:

- BigDatalog - synchronous: There is synchronization after every iteration.
- SociaLite - combination of synchronous and asynchronous: There is synchronization after every stratum, but within a stratum results are immediately shared using message-passing.
- NDlog - asynchronous: A technique called pipelined semi-naïve evaluation (PSN) is used. A newly received tuple is processed by a node immediately. To avoid redundancy of computation, each tuple is given a timestamp. A new tuple can only be combined with other tuples that have the same or an older timestamp.

In Myria, the developers implemented both synchronous and asynchronous evaluation [26]. What they found is that neither was superior over the other. When evaluating a program for computing connected components, they found asynchronous evaluation was more efficient. On the other hand, synchronous evaluation seemed to perform better when computing the least common ancestor for pairs of publications in a citations graph. Finding the best method for the program and data in question depends on the number of iterations and the number of intermediate facts derived. Since these statistics are only known at runtime, this has become quite a difficult optimization problem.

Asynchronous and coordination-free programs have been researched in the context of declarative networking. Researchers looked at how to reach minimal synchronization using Datalog and its extensions. They found some blocking was necessary to some extent for programs with negation (semi-positive Datalog), while normal positive Datalog programs always yielded consistent results regardless of the presence of blocking.

The question arose to what extent blocking and synchronization were necessary in the context of Datalog

semantics. It was hypothesized that a strong relationship existed between program monotonicity and the consistency of program executions. This relation is often referred to as CALM. An exploration of this relation was done by Ameloot, Neven and Van Den Bussche [29]. They found that for a particular form of non-blocking, called coordination freedom, monotone queries have a non-blocking strategy, under the presence of weak network reliability and with no restrictions on data partitioning. In later work [30], it was found that if machines know how data is partitioned, semi-monotone programs can also be executed without blocking, with the underlying idea being that a semi-monotone query can be seen as a monotone query taking as input over the input relations and their complements. Theoretical upper bounds, sadly, do not result in practical algorithms.

3.3.2 Shared-Memory Datalog Engines

Shared-memory Datalog engines are more likely what someone usually thinks of when they hear the term "parallel computing". In this architecture, all nodes receive shares of the workload, but they have shared resources in the form of main memory and disk. RecStep, Datalog-MC [27] and Soufflé use this architecture and are main-memory systems. Graspan also uses this architecture, but it also considers disk movement. It is, however, limited to binary relations, which leads to it not supporting full positive Datalog.

In shared-memory systems, the main concern is handling conflicts over the shared data structures that are accessed each iteration. For this reason, a lot of attention has been paid to constructing data structures and dedicated operators that lend themselves well to concurrency.

Datalog-MC constructs partitions using hash-partitioning. The selection of attributes to base the partitioning on - called the discriminating set - is the task of an optimizer and is based on the query workload and the estimated cost of the best plan for each plan in the query workload. Rules are represented as 'and-or' trees, with an 'and' representing a join operator and an 'or' representing a filter operation or a union. When it comes to evaluating the 'and-or' trees, this happens bottom-up, by sending facts (leaves) upwards to operators, of which the result gets passed up further, until a result is computed at the root. The evaluation is done in parallel by creating multiple copies of this tree that can be evaluated at the same time. This allows for careful reasoning on whether a read or write lock is needed.

RecStep relies on a very efficient relational database management system called QuickStep. To use this technique, RecStep needs to change the rules into SQL queries, which it then passes along to QuickStep for optimization and execution. QuickStep has high-performance parallel operators, with extra attention for the deduplication operator. The deduplication operator is used on every IDB relation during every iteration, and for this reason, it must be highly optimized. Otherwise, it will cause a major bottleneck. For this deduplication operator, RecStep utilizes a global latch-free hash table, where tuples can be inserted by every partition in parallel. The optimizer also ensures this table uses as many buckets as the main memory allows, which results in avoiding memory contention.

Soufflé's parallelization is present in the execution of its nested-loop joins over the atoms of a rule. This implies that Soufflé is required to be able to concurrently perform the operation of inserting facts (and thus also checking if the fact already exists) into a relation. For this purpose, Soufflé uses a specialized data structure made to efficiently allow this concurrency.

Graspan partitions its data based on the source vertices. Every iteration, two partitions are joined in main memory to produce more facts, which can be done in parallel. If a partition's size becomes too large, then it is dynamically repartitioned. Graspan's architecture is uniquely suited to its goal of minimizing memory-disk movement, which it does not share with other engines.

Other engines have also performed parallelization based on GPUs. This technique has become quite popular in many fields. More information can be found in [28]. One of the main bottlenecks with this method lies in the deduplication that occurs after every relational operation, which is necessary as the simple data structure (array of tuples) allows for duplicate facts. Another challenge to pay attention to in this kind of system is the minimization of data transfer between the CPU and GPU.

3.4 Compilation vs. Interpretation

Another design aspect to be considered is whether the Datalog program should be compiled or interpreted. There are some interesting facets to this choice as the evaluation of a Datalog program is a dynamic process

R		
X	Y	Z
0	1	1
0	3	1
2	1	1

Figure 3.3: Example data for showcasing the data structures in section 3.5.

with some information only becoming known at runtime, such as the number of iterations or the optimal query plan for a rule. For this reason, when using compilation, the amount of optimizations that can happen during execution becomes limited. Soufflé and SociaLite utilize compilation, to C++ and Java respectively. On the other side, there are RecStep and BigDatalog, which work based on QuickStep and modified Apache Spark respectively.

3.5 Data Layouts

Most Datalog engines use the classic row-oriented data representation. One of its main advantages is easy interfacing with any underlying relational database management system, but as previously mentioned a number of times, some Datalog engines use specialized data structures to enhance the efficiency of their evaluation. Often this is enabled because of a focus on a specific application domain. These data layouts are often able to provide their advantages because they can take advantage of certain properties of the input to either save space or enhance performance.

For the purpose of intuitively showcasing the data layouts, we will use the example data displayed in figure 3.3 as row-oriented data. The data is of a relation R with attributes X, Y, and Z.

3.5.1 Binary Decision Diagrams

The Datalog engine bddbddb uses binary decision diagrams (BDDs) for the purpose of performing pointer analysis. For some types of pointer analysis, the IDB relations we are trying to compute will grow exponentially with the size of the code. BDDs are specialized in such a way that they save more space when storing the data, which leads to improved performance. To import a relation into the BDD data structure, there are a few different steps:

1. A binary encoding must be performed on the relation: Assume the values for attribute X are from a domain D_X, which contains values $\{0, 1, ..., n\}$. Then we can encode any value from X using a maximum of $log_2(n)$ bits. After computing the necessary number of bits for each attribute, we can compute how many bits are necessary to represent an entire tuple. Let's assume that the domains for our example data are the following: $D_A = \{0, 1, 2, 3\}$, $D_B = \{0, 1, 2, 3\}$, $D_C = \{0, 1\}$. Then to represent a value from attribute A we would need 2 bits. The same number of bits is necessary for B and only 1 bit is necessary for attribute C. In total that gives us 5 bits for a tuple from relation R. The binary encoding of our example data can be seen in figure 3.4.

2. At this point, a database instance can be interpreted as a boolean function f with a mapping from the cartesian product of all domains of our attributes (in our example $D_A \times D_B \times D_C$) to the set $\{0, 1\}$. Now, for a tuple t, $f(t) = 1$ if t is in the given database instance. Otherwise, $f(t) = 0$. A visual representation of f for our example data an be seen in figure 3.5.

A binary decision diagram is a directed acyclic graph, with the key features being the fact that it has a single root node and two terminal nodes: 0 and 1. Every non-terminal node is a bit from a binary encoding of a tuple t and is used as an input variable. Each non-terminal node has two outgoing edges: a 0-edge and a 1-edge. The 0-edge is followed if the provided input bit is a 0 and conversely, the 1-edge is followed if the input bit is a 1. To evaluate a given input on a binary decision diagram, we start at the root node and follow the edges based on the corresponding variables at that point in the graph until a leaf node is reached. The visualization in figure 3.4 is thus the BDD for our example data.

R				
X		Y		Z
0	0	0	1	1
0	0	1	1	1
1	0	0	1	1

Figure 3.4: Binary encoding of the data in figure 3.3.

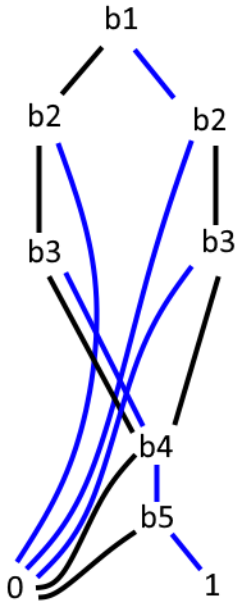

Figure 3.5: Binary decision diagram of the data in figure 3.3. Black edges are 0-edges. Blue edges are 1-edges.

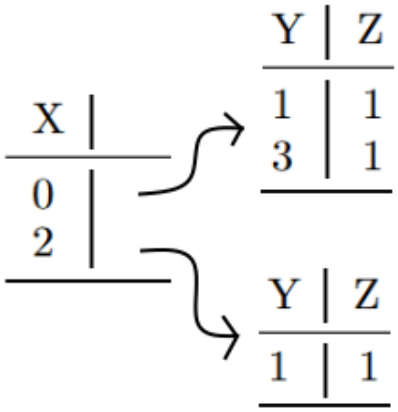

Figure 3.6: Tail-nested table representation of the data in figure 3.3.

Datalog engine bddbddb utilizes a special version of this data structure called ordered binary decision diagrams. The restriction this variant provides is that the order of the variables is consistent no matter which path is followed through the OBDD. This order has a significant influence on how compact the BDD is, but the computation of the ideal order is NP-hard. For this reason, bddbddb uses heuristics to select a good order. Furthermore, during the construction of the BDD, common subgraphs are collapsed and shared so nodes can be reused. It follows that the more redundant the original BDD is, the more space-efficient this representation becomes. It is because of this characteristic that the BDD can be so efficient (both for space and runtime).

3.5.2 Tail-nested Tables

Tail-nested tables are a data structure that is specifically designed for graph representations. SociaLite primarily uses this data structure as the engine was tailored for network analytics. The idea underlying tail-nested tables is that an adjacency list can be used to represent graphs.

In this data structure, the last column in the table contains a variable pointing to another tail-nested table (except for the deepest table, which is simply a two-dimensional table without pointers). The level of nesting is arbitrary, with the structure denoted by the use of parentheses in the table declarations. The relation of our example data would be represented as follows: $R(A, (B, C))$.

If we want to translate this data structure to a graph representation, then A would be the source of an edge, B would be the target of that edge, and C a property of the edge or the target node. A visualization of the data layout can be seen in figure 3.6. Tail-nested tables have two main advantages when used for graph representation:

- Edges with the same source node need to store the source node only once, which provides compact storage.
- The data representation acts as an index to enhance evaluation performance.

3.5.3 Tries

A trie is a data structure in a similar vein as tail-nested tables. A trie is a tree-based data structure in which each level corresponds to an attribute of the relation, and each tuple is represented by a unique path from the root to a leaf node. An example can be seen in figure 3.7. To accelerate any search operation, the children of each node in the trie can be arranged according to a fixed order that is consistently applied across the corresponding level. Just like with OBDDs, tries need a certain order of attributes. This order will influence the efficiency of program evaluation. Finding the optimal ordering is a difficult optimization problem. Tries provide compact data storage, similarly to tail-nested tables. They are also quite efficient for evaluating non-recursive Datalog programs with worst-case optimal algorithms.

Tries see use in various engines. They are used in LogicBlox as a virtual abstraction over other data structures. A variant of tries also sees use in Soufflé. This variant is called the brie and it is a specialized concurrent data structure that combines characteristics of tries and B-trees. It is particularly efficient

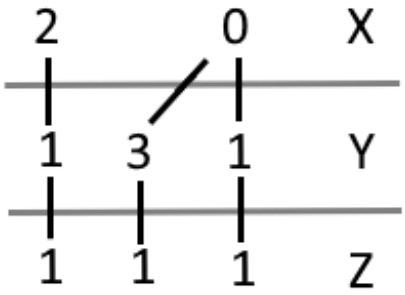

Figure 3.7: Trie representation of the data in figure 3.3.

		Y			
		0	1	0	1
X	0	0	0	0	0
	1	0	1	0	0
	2	0	0	0	0

Figure 3.8: Bit matrix representation of the data in figure 3.3.

when used to store highly dense relations. This makes sense when you realize that highly dense relations will have a large number of tuples that have sizable overlap, which can be compactly represented using bries.

3.5.4 Bit Matrices

Being able to represent dense relationships seems to be a popular characteristic to have because it often happens that even small EDB relations can result in large and dense relations. An example program where this is the case is the program we have referenced numerous times, the program for computing transitive closures. Even if the input graph has $O(n)$ edges, with n being the number of nodes, then the output relation can have $O(n^2)$ edges. Normal row-oriented storage quickly deteriorates in efficiency because of costly joins and perhaps even extra I/O overhead if the relation grows too large for main memory.

This is where RecStep introduces Bit matrices, which is another good data structure for representing dense relationships. A bit matrix is constructed by making an n-dimensional matrix, with n being the number of attributes in a relation R, where the dimensions of the matrix are equal to the sizes of the active domains of the attributes. For our example data, we would end up with a $4 \times 4 \times 2$ matrix. In this matrix, every tuple is represented using a single bit. For every dimension of the matrix, the corresponding value of the tuple is used as an index to navigate the matrix. If the bit of a matrix is 1, then the tuple with the corresponding indices as values exists in the database. An example of a bit matrix can be seen in figure 3.8.

Another advantage of the bit matrix representation is that it is easy to update by simply changing the bits corresponding to new tuples to a 1. Furthermore, the space of the representation remains constant throughout the entire evaluation. One final benefit we note here is the fact that it lends itself well to parallelization by partitioning the data structure row-wise and providing each thread with partitions.

3.6 Indexes

After considering which data layout to use, another design consideration that must be made is what indexes to utilize. Classic indexes such as hash-tables and B-trees definitely enhance performance because most engines perform relational data processing. Nonetheless, some Datalog engines still opt to make use of specialized indexes.

One such example of a specialized index can be seen in Soufflé. There is an interesting separation of read and write operations when it comes to synchronous evaluation. In synchronous evaluation, read operations are always followed by various write operations, though this does not seem to be the case for asynchronous evaluation. Soufflé benefits from this separation by utilizing a special B-tree with an optimistic fine-grained locking scheme [31].

A substantial amount of research exists on index recommendation for relational database management systems, the majority of which rely on heuristic techniques. Sadly, these techniques fall short for evaluating Datalog programs for numerous reasons:

- Indexes are necessary for all types of relations, but for IDB relations these indexes need to be dynamic since the relations change over the evaluation.
- Techniques that do not scale, will fail for large Datalog programs.

Most Datalog engines opt for easier solutions, such as simple strategies or even letting the user decide on what indexes to use (SociaLite). RecStep does not build indexes upfront, but the underlying RDBMS creates hash indexes for the join and deduplication operator. Engines that use indexes based on an order of attributes, think of tries and BDDs, also need to consider the optimal attribute order, which is usually done based on heuristics and collected statistics. Soufflé does opt to build all its B-tree indexes ahead of time. It considers the rules it has to evaluate and then finds a subset of indices that can be used to speed up all nested-loop joins. Because of its more restrictive setting, this process can be executed in polynomial time. However, a major disadvantage to consider when reviewing Soufflé's design choice here is the memory necessary to store all the indexes, which can ramp up when a large number of indexes are constructed. This problem was considered during the development of the automatic index selection technique, as the subset of indices returned is minimal. For a deeper dive into this topic, we refer the reader to Subotić et al. [49].

3.7 Other Optimization Techniques

3.7.1 Magic Set Transformation

As we have gone deeper into the magic set transformation in 2.2.3, we will stay shallow here. The magic set transformation is a language-level transformation, which means it is an optimization technique that works by producing a rewritten version of the program. It, or a restricted variation of it, has been implemented by some engines, such as Soufflé and SociaLite. It works by pushing the query selection into the rules of the program. The result of this is that only the facts necessary to derive the desired result are computed. This restriction of the derivation of facts is what enables the time savings compared to the evaluation of the original Datalog program.

3.7.2 Reduction in Number of Iterations

In synchronous computation, every iteration is equivalent to a synchronization barrier. To avoid these synchronization barriers from becoming a bottleneck, some Datalog engines try to reduce the number of iterations. It is clear how this could speed up parallel evaluation. Unfortunately, reducing the number of iterations often goes paired with increasing the average number of derived tuples. So there is a trade-off between synchronization and communication. An example of the technique applied to an example can be seen in figure 3.9.

```
Tc(x, y) :- Arc(x, y).
Tc(x, z) :- Tc(x, y), Tc(y, z).
```

Figure 3.9: Rewritten version of transitive closure program from figure 2.2 to have a logarithmic number of iterations in relation to the input size.

Research has been done into finding if this decrease in the number of iterations could happen without causing an increase in derived tuples [32]. They found this was possible for some Datalog programs (such as transitive closure), but not for others. This technique has not been implemented in a Datalog engine as of yet.

3.7.3 Reduction in Number of Strata

Synchronization barriers can also occur in asynchronous evaluation in the presence of stratified negation. Similar to the last technique, we want to reduce the number of synchronization barriers (in this case, the number of strata) to enhance the performance of parallel evaluation.

The idea underlying some proposed techniques [33, 34] involves combining an unrolling of a program's recursion with checking whether any subset of negated atoms can be safely eliminated without altering the program's semantics. Similarly to the techniques for reducing the number of iterations, these techniques have not yet been implemented in any Datalog engines.

3.7.4 Pushing aggregation into recursion

Rewrites can also be of use to speed up recursive aggregation in a program. The idea is similar to magic set rewriting: to push aggregation into the recursion. Research has already been done into this for certain types of aggregation [35–37]. An example can be seen with figure 3.10 being the original program and figure 3.11 being the rewritten program. The added aggregation in the first two rules allows for fewer deduced facts, leading to a smaller $Path$ relation. This, in turn, speeds up the evaluation of the rest of the programs since joins will be executed more quickly. It is also trivial to see that the added aggregations stem from the fact that the $SPath$ relation is computed by applying an aggregation to the $Path$ in one iteration. Thus it is not far-fetched to already apply this aggregation during the earlier stratum.

```
Path(x, cost) :- Edge('a', x, cost).
Path(x, cost) :- Path(y, cost'), Edge(y, x, cost"), cost = cost' + cost".
SPath(x, min(cost)) :- Path(x, cost).
```

Figure 3.10: Datalog program for computing the cost of the shortest paths from a to all other nodes.

```
Path(x, min(cost)) :- Edge('a', x, cost).
Path(x, min(cost)) :- Path(y, cost'), Edge(y, x, cost"), cost = cost' + cost".
SPath(x, min(cost)) :- Path(x, cost).
```

Figure 3.11: Datalog program for computing the cost of the shortest paths from a to all other nodes, but rewritten so aggregation is pushed into the recursion.

In original research [36], it was shown that this transformation can be executed properly for a specific condition called the PREM property. In a more recent study [38], this transformation was generalized by utilizing program synthesis techniques.

3.7.5 Search Space Extension for Plans

New possibilities for query plan optimization have been discovered by combining relational algebra and recursion. Examples of such techniques are described in [39]. These techniques provide alternate execution strategies for a subset of Datalog that coincides with relational algebra combined with a fixpoint operator. This algebraic formulation allows the authors to derive novel rewriting rules that produce new query execution plans. A rewrite rule can push a filter inside a fixpoint computation or combine two fixpoint computations.

For an example, we consider the Datalog program of figure 3.12. Relation U in this program will compute all paths consisting of edges in relation R followed by edges in relation S. Semi-naive evaluation will first compute T to then compute U, but with the new techniques we can compute the join of relations R and S as base facts for relation U and then extend U on the left or right of the join using facts from R and S respectively.

A simple cost-based optimizer determines the best plan and executes it. As of yet, this has not been implemented in a Datalog engine.

```
T(x,y) :- R(x,y).
T(x,y) :- T(x,z), R(z,y).
U(x,y) :- T(x,z), S(z,y).
U(x,y) :- U(x,z), S(z,y).
```

Figure 3.12: Datalog program for computing all paths consisting of a sequence of edges from relation R followed by a sequence of edges from relation S.

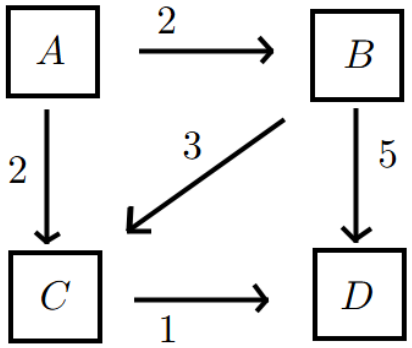

Figure 3.13: Directed graph for delta stepping example.

3.7.6 Delta Stepping

The last optimization technique we discuss here is 'delta stepping'. SociaLite utilizes this technique to enhance performance when evaluating recursive monotone aggregate functions. This technique originates in shortest-path computation.

We try to provide an intuitive understanding by means of an example. Consider an aggregate function such as 'min'. This function will converge faster if it only needs to operate on the smallest values. Delta stepping achieves this by simultaneously updating multiple 'minimum' values during asynchronous evaluation. This way, it avoids having to harm the asynchronous nature of SociaLite by inserting synchronization barriers by means of a minimum aggregate.

Assume we want to compute the shortest paths from node A in the graph from figure 3.13 and let us assume a delta of 2, so bucket 0 will contain the nodes with a path cost of $[0, 2)$, bucket 1 will contain the nodes with a path cost of $[2, 4)$, and so on. Bucket 0 will contain A. Since node A is where we start, we give it a path cost of 0. After processing A, we know both B and C have a cost of 2, so both are placed in bucket 1. Since bucket 0 is empty now, we move on to bucket 1. Both B and C can now be processed in parallel. Processing C will give us a shortest path of 3 for D, so D is inserted into bucket 1 and processed as soon as possible.

Chapter 4

Implementation Architecture

To evaluate the performance characteristics of recursive query processing and the effectiveness of optimization techniques, a custom implementation was developed as the foundation for empirical analysis. The architecture aims to support both naïve and optimized evaluation strategies and hopes to facilitate direct comparisons in terms of execution time and scalability across multiple different recursive queries. This chapter describes the architectural design and key components of this implementation. It also discusses design choices, data structures, and the execution pipeline employed in the system.

This chapter provides the necessary context for interpreting the empirical results showcased in a later section, by describing the internal structure and logic of the custom implementation.

4.1 Rust

The implementation of the custom Datalog engine was built on top of the increasingly popular programming language 'Rust'. This language has been dubbed "The most loved programming language by developers" on Stack Overflow. There are many good qualities of Rust that earn it this title.

First, Rust's standard libraries are widely recognized for their comprehensive documentation. Rust in general also encourages good documentation. Among other things, through the use of the built-in tool called 'rustdoc', which creates documentation from code comments. This results in many community-produced crates also possessing a high level of quality when it comes to documentation. This documentation is visible on a well-formatted docs page which automatically gets constructed from your personal docs when publishing a crate.

Second, Rust has a central repository (crates.io) for its large collection of community-produced crates. Oftentimes, good implementations for functionalities can be found in this repository. Furthermore, all versions of these crates are maintained and can be pulled. Through the use of 'cargo' (Rust's package manager, build system, and dependency resolver), these crates can be easily fetched and compiled for use in projects.

Third, Rust provides excellent performance while maintaining code safety and ease of use. It does this through a few techniques:

- Zero-cost abstractions: Iterators, closures, pattern matching and many other functionalities are compiled down to performant machine code with no overhead compared to implementations using a lower-level language like C. This allows users to write high-level expressive code without sacrificing performance.
- Ownership and borrowing system: Rust's ownership model ensures safe memory usage without the use of a garbage collector. This results in no performance-cost of programs caused by the garbage collection process and safe memory usage enforced at compile time.
- Compile-time guarantees: Ensures there are no data races, no null pointer dereferencing and no buffer overflows. Moreover, it ensures strict lifetimes and borrowing rules so unsafe code is rare and isolated.

- Efficient data structures + system calls: Rust lets you work directly with memory and system calls like in C/C++.

Lastly, Rust supports concurrency through the use of specialized data structures (e.g. Arc) and features (e.g.). There are also safety guarantees with parallelism in mind, such as data races being a compile-time error. There are also crates created for the express purpose of handling parallelism. One such crate is 'Tokio', which makes parallelism easy to use and provides a simple interface through the 'async' and 'await' keywords.

All these aspects make Rust a good candidate for the foundation of the custom implementation. As an extra bonus, Rust is easy to learn and understand through the many tools that the developer team provides, such as:

- The extensive official book "The Rust programming language", which provides a clear breakdown of all aspects in rust including examples and small projects to see the concepts in use.
- The 'rustlings' course, which guides you through setup and teaches you through a large list of exercises to test your knowledge.

4.2 DataFusion

Apache DataFusion [50] is an extensible, open-source, in-memory, columnar data management system and query engine. It was written in Rust for the safety, speed, and concurrency benefits Rust provides. The fact that Apache DataFusion is an easily usable crate to import is another reason for our choice of Rust as our programming language. Apache DataFusion uses Apache Arrow as its in-memory format.

We utilize Apache DataFusion because it provides us with basic implementations of a plethora of components we need to build our custom Datalog engine. This way we do not need to start from scratch. It also provides fine-grained access to the construction of logical plans for our queries. Furthermore, Apache DataFusion has a built-in optimizer for logical plans that we can utilize before transforming our logical plans into execution plans to improve performance. Other performance-oriented features Apache DataFusion provides include optimizations such as: projection pushdown, multi-threaded query plan execution through the use of Rust's Tokio crate, et cetera.

Apache DataFusion works with Parquet and csv files out of the box. We opt for the former, which is a columnar binary format, for several reasons:

- The parquet file format utilizes compression to store the data using less memory space.
- Parquet files store data types and schema metadata.
- Parquet files are more efficient for reading large datasets.

Naturally, there are also some disadvantages:

- Parquet files are difficult to edit.
- Parquet files are not human-readable

These disadvantages are negligible as we do not operate directly on parquet files anyway. The most problematic this could be is when debugging, if the developer wants a small, flexible data file. If this is deemed really important, then the user could opt to use csv during testing/debugging and then switch to using parquet files for the final version. Furthermore, while it might not be possible to do these things in the parquet files themselves, it is still possible through Apache DataFusion, which further diminishes the effect of these disadvantages.

For those interested, Apache DataFusion offers a lot more benefits that were not necessary in the custom Datalog engine implementation for this thesis. One of these benefits is Apache DataFusion's extensibility. Because of the modular architecture of Apache DataFusion, it is possible to add extra planner rules, custom physical execution operators, or specialized data formats.

4.3 Pre-processing

The custom implementation expects various different files as input arguments. In the next section, we briefly describe any pre-processing necessary to acquire this data in the right file format.

4.3.1 Datalog Programs

The representation of a Datalog program within the custom implementation is through a struct containing a vector for EDB declarations, a vector for IDB declarations, a vector for Rules, and a vector for representing the strata. EDB declarations, IDB declarations, and Rules are in turn composed of other substructs and fields.

The necessary information for a program was provided at runtime in a JSON format. These JSON formats were constructed using a Python script. This Python script relied on some Python classes, which are similar to the Rust structs previously mentioned. The information was extracted from the RecStep Python representation of Datalog programs, which can be found on their GitHub page [41].

This JSON format is imported into Rust using the Serde crate from Rust. Serde is a framework for serializing and deserializing Rust data structures efficiently and generically. Serde is built on Rust's powerful trait system in order to avoid relying on runtime reflection for the (de)serialization process. This way we can avoid the overhead that usually comes with this process. We specifically employ Serde's JSON API. Being able to use Serde + JSON saves us from having to implement our own data format, serializer, and deserializer for transferring the Datalog Programs from their original format to the implemented Rust structs.

4.3.2 Data Files

The data files (which will be discussed in a later section) also need to be transformed into parquet files for use as EDB relationships in Apache DataFusion. First, we edited away the header manually where necessary. Afterward, we utilized the Pandas library for Python. Next, we read the data into a pandas DataFrame as a CSV formatted file and removed any unnecessary columns where needed. Lastly, we used pandas to write the DataFrame to a parquet file, making sure not to add a header or an index column.

4.4 Implementation Logic

In the next section, we describe how the custom Datalog engine implementation handles the evaluation of a Datalog program.

1. The implementation starts by checking if all the necessary arguments have been provided, these being the Datalog program JSON, a query text file, and the folder path where the parquet files are located for the EDB predicates. The default evaluation method is naive evaluation, but an optional flag can be passed to the implementation as an argument to change this to semi-naive evaluation or semi-naive evaluation with the magic set rewriting technique.
2. We create a session context in Apache DataFusion, necessary for registering tables, executing plans, et cetera.
3. A dictionary is created containing logical plans that refer to the relations.
4. The next step is to import the Datalog program (IDB relations, EDB relations, rules, and rule order) using Serde and read the query to obtain the binding variable. The latter of which only really has an effect when applying the magic sets rewrite technique.
5. The EDB and IDB declarations in the Datalog program are processed, and tables are created in the session with the correct schema (and values for the EDB predicates).
6. Now, the code splits up into 3 different branches, but first, we provide a brief rundown of some of the data structures used in this implementation. We have tables that we can access saved in the session context. However, most of the time we use logical plans as an abstraction for these tables as this is easier to work with. These are usually saved in dictionaries with the table names acting as keys.

- Naive evaluation:
  (a) For every stratum we execute the steps underneath.
  (b) We start a loop l.
  (c) Create a vector v to keep track of if a rule was able to derive any new tuples.
  (d) We copy our current logical plans.
  (e) For every rule in the stratum we do the following:
    i. We create a query plan for the current rule by simply joining all the predicates from left to right.
    ii. Project the variables from the head of the rule out of the variables from the relation resulting from the join of body predicates.
    iii. Perform the union of the newly derived tuples and whichever tuples are in the database for the relation in the head of the rule.
    iv. Replace the table in the database with the result of the plan of the union mentioned in the previous step.
    v. Check if the logical plan for the table from the previous iteration is different from the logical plan for the current table and save the resulting boolean in vector v.
  (f) If not a single boolean in vector v indicates that a table has changed, then break out of loop l.
- Semi-naive evaluation:
  (a) For every stratum, we execute the steps underneath.
  (b) Create an extra dictionary to hold the logical plans that compute all delta tables.
  (c) Copy our current logical plans that compute all tables (this will be the dictionary for holding the logical plans that compute the tables of the previous iteration).
  (d) We start a loop l.
  (e) Create a dictionary for holding the logical plans of the delta tables for the next iteration.
  (f) Create a vector v to keep track of whether a rule was able to derive any new tuples.
  (g) For every rule in the stratum, we execute the following steps:
    i. We create a query plan for the current rule by following the semi-naive rule evaluation which was explained in detail in section 2.2.1.
    ii. Project the variables from the head of the rule out of the variables from the relation resulting from the query plan.
    iii. Extract the delta tuples from the newly-derived tuples by performing a difference between the newly-derived tuples and the tuples already in the database.
    iv. Perform the union of deltas just derived with any deltas already computed for this IDB relation this iteration.
    v. Check if the delta for the rule is empty and save the resulting variable in vector v.
  (h) If not a single boolean in v indicates that a relation will have new tuples, then break out of loop l.
  (i) Update our dictionary holding the logical plans that compute all of the tables of the previous iteration.
  (j) Update our tables for the deltas with the result of the logical plan that computes the new deltas.
  (k) Update our tables for our complete relations by computing the union between our deltas and our existing tables.

- Magic sets rewrite + semi-naive evaluation:
  (a) Perform the magic-sets rewrite technique:
    i. Create the input relations for the different IDB relations.
    ii. Populate the input relation for the relation in the query with the bindings provided by the query.
    iii. For every rule in the original program, execute the following steps:
      A. Create a rule for the first supplementary relation using input relation of the relation in the head of the original rule.
      B. Create a rule for every supplementary relation i by joining the previous supplementary relation $i-1$ of this rule with the predicate at index $i-1$ in the original rule.
      C. For the supplementary relation rule that would serve as the answer set for the original rule, replace the head with the rule head of the original rule.
      D. For all rules computed, if any of the predicates is an IDB predicate, construct a rule for updating the corresponding input relation with the correct variables from the supplementary relation that IDB predicate is being joined with.
    iv. With all rules now constructed, determine the stratum of every relation by repeatedly computing using the following rules: For a predicate A in the head and a predicate B in the body
      – if a variable x from A is a non-aggregate variable and x occurs in B, then the stratum of A has to be larger or equal to the stratum of $B \rightarrow$ set stratum of A equal to stratum of B, if it was originally smaller.
      – if a variable x from A is an aggregate variable and x occurs in B, then the stratum of A has to be larger than the stratum of $B \rightarrow$ set stratum of A equal to stratum of B incremented by 1, if it was originally smaller.
  (b) Perform the new program using semi-naive evaluation.
7. The relations resulting from the evaluation are now optionally tested against a "truth" provided by the user by checking if one relation is a subset of the other and the inverse.
8. Execution times are optionally stored for later referencing.

Chapter 5

Comparison

To properly evaluate the influence of specific techniques on the efficiency of a Datalog engine, we must rely on empirical results. In this chapter, we describe the comparison methodology, including the Datalog engines used for evaluation, the benchmark datasets and programs applied, and a discussion of the performance results for all experiments.

5.1 Experimental setup

Our benchmarks were run locally on a machine with 16 GB of RAM and a single Intel i9-10980HK 2.40GHz processor with 8 physical cores and 16 logical cores. The custom implementation was tested in Windows 10 PowerShell with 16 threads, as Rust Tokio defaults the number of threads to the number of logical cores. RecStep and Soufflé were tested in WSL Ubuntu 20.04.6 LTS. RecStep was tested on the last available version on GitHub as of February 2025 with 40 threads. Soufflé was tested on version 2.4 with the default threading number, which is equal to the number of logical cores. The times for RecStep were measured using the interpretation of the Datalog program (without the use of the PBME technique mentioned in Fan et al. [23]), while the times for Soufflé were measured by running the binary resulting from compiling the Datalog program using the -o flag.

5.1.1 Engines

The custom implementation for this thesis supports three different evaluation modes, as previously described: naive evaluation, semi-naive evaluation, and a combination of magic-set rewriting with semi-naive evaluation. These modes are compared to each other to measure the performance benefits in relation to the runtime of each optimization technique. In addition to this comparison, the three modes of the implementation are also evaluated against two other state-of-the-art engines. For the engines in question, we chose RecStep [23] and Soufflé [22].

RecStep [23] is designed to be a high-performance, widely applicable Datalog evaluation engine, particularly for large workloads. It reportedly scales well across multiple cores and large datasets, and it has seamless support for all manners of recursive queries. Proof of RecStep's excellent performance is provided in Fan et al. [42], section 6 where we can observe in figures 8 through 16 that RecStep is competitive with other highly optimized and specialized Datalog solvers.

Soufflé [22] enjoys widespread use in both academic and industrial fields, especially in program analysis. It has also proven to be quite performant over its lifespan and in academic papers. It compiles Datalog into highly optimized C++ code, offering a different optimization approach compared to the interpreter approach of RecStep and the custom implementation.

By comparing with these engines, our comparison gains broader relevance because of the two different design philosophies used in these Datalog engines: RecStep focuses on multicore runtime optimization, while Soufflé emphasizes ahead-of-time compilation.

Transitive closure (TC)

$$Tc(x, y) : -Arc(x, y).$$

$$Tc(x, y) : -Tc(x, y), Arc(y, z).$$

Figure 5.1: Datalog program for computing transitive closures in a graph.

Reachability (Reach)

$$Reach(y) : -Id(y).$$

$$Reach(y) : -Reach(x), Arc(x, y).$$

Figure 5.2: Datalog program for computing Reachability from given nodes in a graph.

5.1.2 Programs

As benchmark Datalog programs, we use a subset of the programs described in the sixth section of the RecStep paper [42]. These programs originate from two fields: graph analytics and static program analysis. We provide a description of every program written in Datalog in figures 5.1, 5.2, 5.3, 5.4, 5.5, 5.6.

During the execution of the benchmark tests, we noticed that Soufflé is not able to evaluate the basic connected components Datalog program. This is because of the recursive aggregation present in the program which Soufflé cannot stratify. For this reason, we decided to add another connected components program, found in 'Modern Datalog Engines' [40] by Bas Ketsman and Paraschos Koutris. Instead of running Soufflé on the alternate program and comparing those results to the results of the other engines of the original program, we ensure a fair evaluation by benchmarking separately on two distinct versions of the connected components program.

Readers comparing this set of programs to the original set of benchmark programs from Fan et al. [42] might notice the absence of the Datalog programs for Andersen's analysis (AA), single source shortest path (SSSP), and same generation (SG). The Datalog programs for single source shortest path and same generation computations were excluded because our custom implementation does not support arithmetic operations when evaluating Datalog programs, which are a vital component present in these two Datalog programs. Consequently, we could not benchmark these. The Datalog program for Andersen's analysis was not included in our set of benchmarks because of an inability to easily locate or construct a dataset with the necessary characteristics to execute a meaningful evaluation.

5.1.3 Datasets

Various datasets were used when benchmarking each program. This is intended to prevent obtaining only results that are overly specific or biased due to particular patterns present in any dataset.

The Datalog programs for static program analysis - these being context-sensitive points-to analysis (CSPA) and context-sensitive dataflow analysis (CSDA), visible in figures 5.5 and 5.6 respectively - were evaluated using datasets based on codebases from Linux, HTTPd, and PostgreSQL. These datasets were also used in Fan et al. [42] and Wang et al. [43] and have thus proved to be qualitative datasets.

The Datalog programs for graph analysis - these being transitive closure (TC), reachability (Reach), and both versions of connected components (CC), visible in figures 5.1, 5.2, 5.3 and 5.4 respectively - were

Connected components (CC)

$$Cc3(x, MIN(x)) : -Arc(x,).$$

$$Cc3(y, MIN(z)) : -Cc3(x, z), Arc(x, y).$$

$$Cc2(x, MIN(y)) : -Cc3(x, y).$$

$$Cc(x) : -Cc2(, x).$$

Figure 5.3: Datalog program for computing connected components in a graph.

Alternate connected components (CC - Bas Ketsman)

$$Connected(x, y) : -Arc(x, y).$$

$$Connected(x, x) : -Arc(x, \_).$$
$$Connected(x, y) : -Connected(x, z), Arc(z, y).$$
$$Connected(x, y) : -Connected(y, x).$$
$$Component(x, MIN(y)) : -Connected(x, y).$$
$$Cc(x) : -Component(\_, x).$$

Figure 5.4: An alternate Datalog program for computing connected components that does not make use of recursive aggregation. From the book 'Modern Datalog Engines' [40]

Context-sensitive Points-to Analysis (CSPA)

$$ValueFlow(y, x) : -Assign(y, x).$$

$$ValueFlow(x, y) : -Assign(x, z), MemoryAlias(z, y).$$
$$ValueFlow(x, y) : -ValueFlow(x, z), ValueFlow(z, y).$$
$$MemoryAlias(x, w) : -Dereference(y, x), ValueAlias(y, z), Dereference(z, w).$$
$$ValueAlias(x, y) : -ValueFlow(z, x), ValueFlow(z, y).$$
$$ValueAlias(x, y) : -ValueFlow(z, x), MemoryAlias(z, w), ValueFlow(w, y).$$
$$ValueFlow(x, x) : -Assign(x, y).$$
$$ValueFlow(x, x) : -Assign(y, x).$$
$$MemoryAlias(x, x) : -Assign(y, x).$$
$$MemoryAlias(x, x) : -Assign(x, y).$$

Figure 5.5: Datalog program for executing a context-sensitive points-to analysis.

Context-Sensitive Dataflow Analysis (CSDA)

$$Null(x, y) : -NullEdge(x, y).$$

$$Null(x, y) : -Null(x, w), Arc(w, y).$$

Figure 5.6: Datalog program for executing a context-sensitive dataflow analysis.

Graph Datasets			
Name	Nodes	Edges	Link
Arabic	163 598	1 747 269	https://networkrepository.com/web-arabic-2005.php
Twitter	304 691	563 069	https://snap.stanford.edu/data/higgs-twitter.html
LiveJournal	3 997 962	34 681 189	https://snap.stanford.edu/data/com-LiveJournal.html
Orkut	3 072 441	117 185 083	https://snap.stanford.edu/data/com-Orkut.html
G2,5k	2 500	6 313	https://www.cse.psu.edu/ kxm85/software/GTgraph/
G5k	5 000	25 046	https://www.cse.psu.edu/ kxm85/software/GTgraph/
G10k	10 000	99 805	https://www.cse.psu.edu/ kxm85/software/GTgraph/

Figure 5.7: Summary of graph datasets: name, number of nodes, number of edges, and website link.

Static Program Datasets					
Name	Nodes	Edges	Assign	Dereference	NullEdge
Linux	42 408 123	44 040 580	1 981 259	7 502 956	589 152
HTTPd	5 744 229	10 043 955	362 799	1 147 612	138 331
PostgreSQL	29 820 407	34 798 150	1 209 597	3 468 946	217 115

Figure 5.8: Summary of the static program datasets datasets: name, number of nodes, number of edges, number of 'Assign' edges, number of 'Dereference' edges, and number of null edges.

evaluated using either randomly generated graphs or large-scale real-world graphs containing over tens of millions of nodes and edges. The Datalog programs for transitive closure and Bas Ketsman's version of connected components were evaluated using the random graphs. Because these programs are highly data-intensive, smaller datasets allow us to benchmark these programs in a reasonable amount of time. The Datalog programs for reachability and the default connected components seem to be less data-intensive, and for that reason, it is feasible to evaluate these programs on the larger real-world datasets.

The randomly generated graphs were created using the GTgraph synthetic graph generator [44]. Three random graphs were made for these benchmarks:

- A random graph with 2500 nodes and a probability of 0.001 for each pair of vertices to be connected in the graph.
- A random graph with 5000 nodes and a probability of 0.001 for each pair of vertices to be connected in the graph.
- A random graph with 10000 nodes and a probability of 0.001 for each pair of vertices to be connected in the graph.

We provide a summary of the datasets with statistics in figures 5.7, and 5.8. To save some space in the summaries for the Linux, HTTPd, and PostgreSQL datasets, we mention here that these datasets are available at the Google Drive [45] corresponding to the Graspan paper [43].

5.2 Benchmarks

For evaluating the magic sets rewriting technique, we sampled 5% of an appropriate dataset to compute our query bindings. This value was chosen as it definitely constrains what needs to be derived while still allowing for significant computation in the case of a medium to large dataset.

For the evaluation of the reachability program, we have chosen a sample of 10% of all node id's to populate the EDB predicate Id with. Taking a sample is necessary to prevent the evaluation of this program from being too simple. If one were to use the full set of node id's, then the first rule of the program would add all these id's to the *Reach* predicate and the second (recursive) rule would not add any other facts since all facts would already be present in *Reach*.

We have imposed an artificial time limit of around 15 minutes on the evaluations executed by our custom implementation for the sake of not taking overly long to collect data. This led to us slightly decreasing the size of some datasets used as EDB predicates. An overview of the sample sizes for the combination

Dataset sample sizes		
Program	Dataset	sampling size
CSDA	HTTPd - Arc	0.90
CSDA	HTTPd - NullEdge	0.90
CSDA	Linux - Arc	0.85
CSDA	Linux - NullEdge	0.85
CSDA	PostgreSQL - Arc	0.90
CSDA	PostgreSQL - NullEdge	0.90
CSPA	PostgreSQL - Assign	0.95
CSPA	PostgreSQL - Dereference	0.95

Figure 5.9: A table displaying the sizes of all samples taken of datasets and for which Datalog programs.

of every dataset and program can be observed in figure 5.9 (if a combination is absent, then no sampling was done).

When analyzing the results for both versions of the connected components program (figures 5.12 and 5.13), it becomes apparent that measurements for the custom implementation using the magic sets rewriting technique are absent. These measurements were deliberately excluded, as programs involving aggregation require a specialized variant of the magic sets transformation to function correctly. Our custom implementation does not support this alternate magic sets technique. As a result, we are unable to include a corresponding custom magic sets evaluation for comparison.

Another absent measurement can be noticed for the connected components program from Bas Ketsman (figure 5.13). The execution for RecStep of this program on the G10k dataset could not conclude because of what we assume to be an out-of-memory error. For this reason, the measurement could not be made and is thus not present.

Furthermore, there are no measurements for CSPA because the computer running the program crashed during multiple attempts. However, for each attempt, it took longer than 20 minutes before the crash occurred. Hence, we assume that a successful execution of the program would take significantly longer than any of the other engines measured.

We note that one must take care when interpreting the results in figures 5.10 through 5.14. A logarithmic scale was used for the axis representing the runtime. This was done for the sake of readability, but as a result, the bars might not give an instinctive representation of exactly how long the evaluations took to complete compared to one another.

In figure 5.16, we also benchmarked results for the custom implementation using magic set rewriting techniques on the transitive closure Datalog program with varying sizes for the dataset containing the query bindings to analyze the influence of this factor. The decimal numbers in this graph signify the sample percentage used to construct our set of query bindings. For ease of comparison, the results of the semi-naïve evaluation of the custom implementation were also added.

5.2.1 Findings

A first thing we notice is that the custom implementation of naïve evaluation is not always the worst. Though when this is the case, it is specifically for datasets where evaluation runtime is so short that optimization will not make that large of a difference overall (Arabic and Twitter datasets). For these datasets, the evaluation will benefit from an absence of overhead, which is naturally the case for naïve evaluation because of its simplicity.

We do note that it is surprising that the naïve evaluation seems to stay competitive with Soufflé and RecStep for the combination of the reachability program and the Orkut dataset. This is quite surprising as there is quite a large difference in size between the Reach relationship and the relationship for the corresponding deltas in the later iterations of the evaluation. This difference is visible in figure 5.17. A possible reason is that the query plan derived by the Apache Datafusion optimizer is coincidentally quite effective, which results in RecStep and Soufflé's optimizations giving less of an edge over the custom implementation than usual, while making their flaws seem more significant. This would also explain

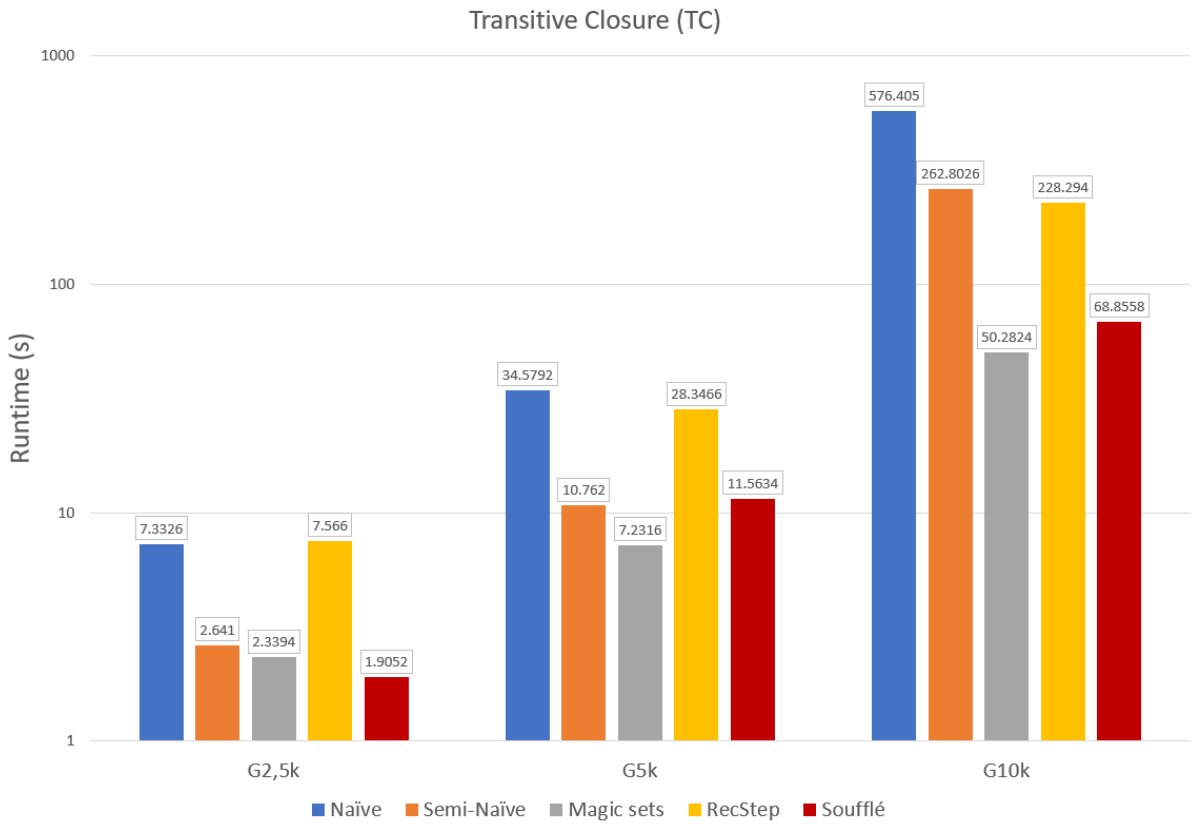

Figure 5.10: Average runtime for every evaluation engine per dataset when evaluating the program for computing transitive closures (figure 5.1).

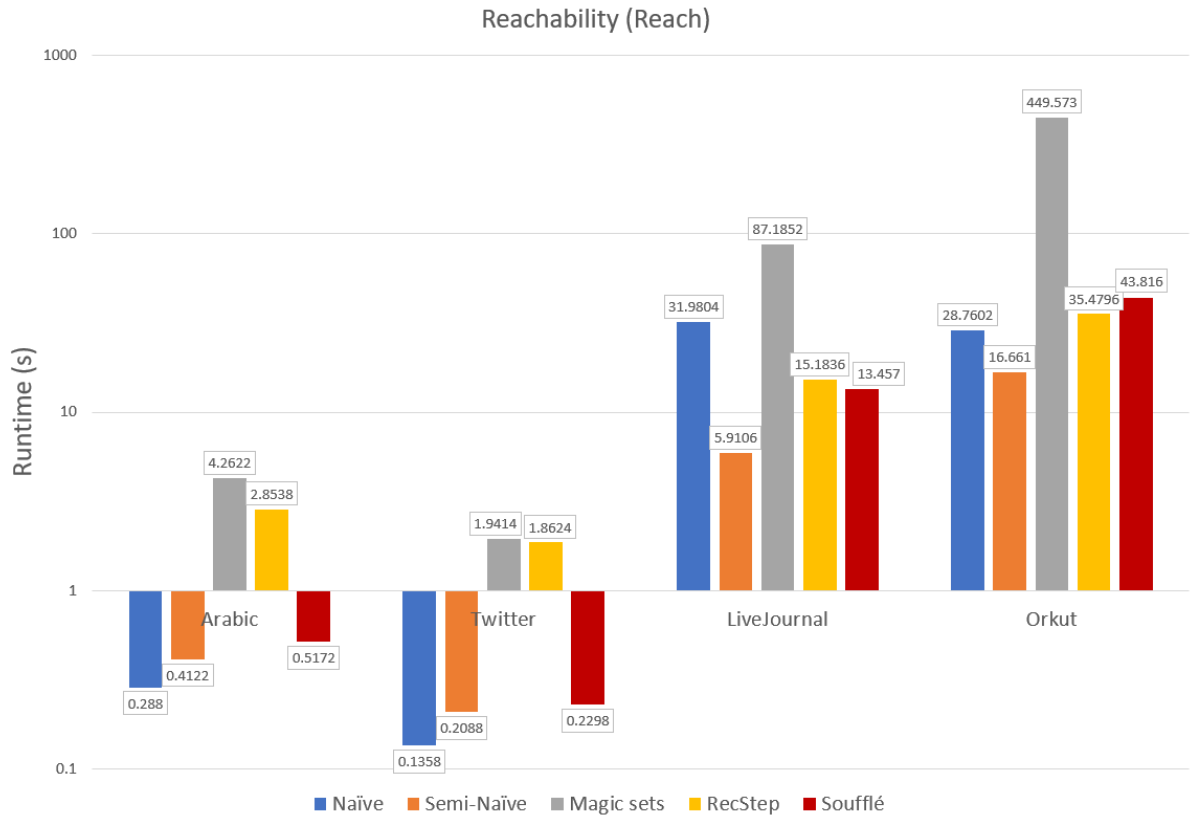

Figure 5.11: Average runtime for every evaluation engine per dataset when evaluating the program for computing reachability (figure 5.2).

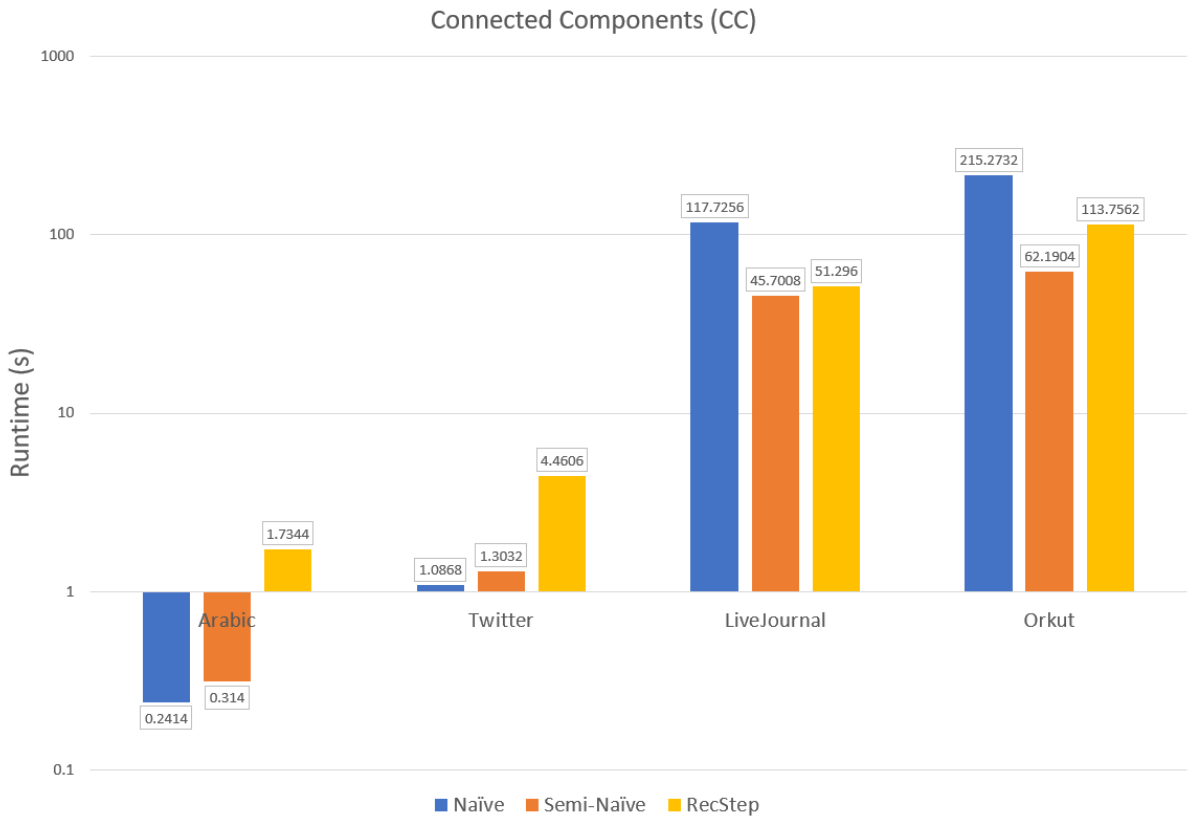

Figure 5.12: Average runtime for every evaluation engine per dataset when evaluating the program for computing connected components (figure 5.3).

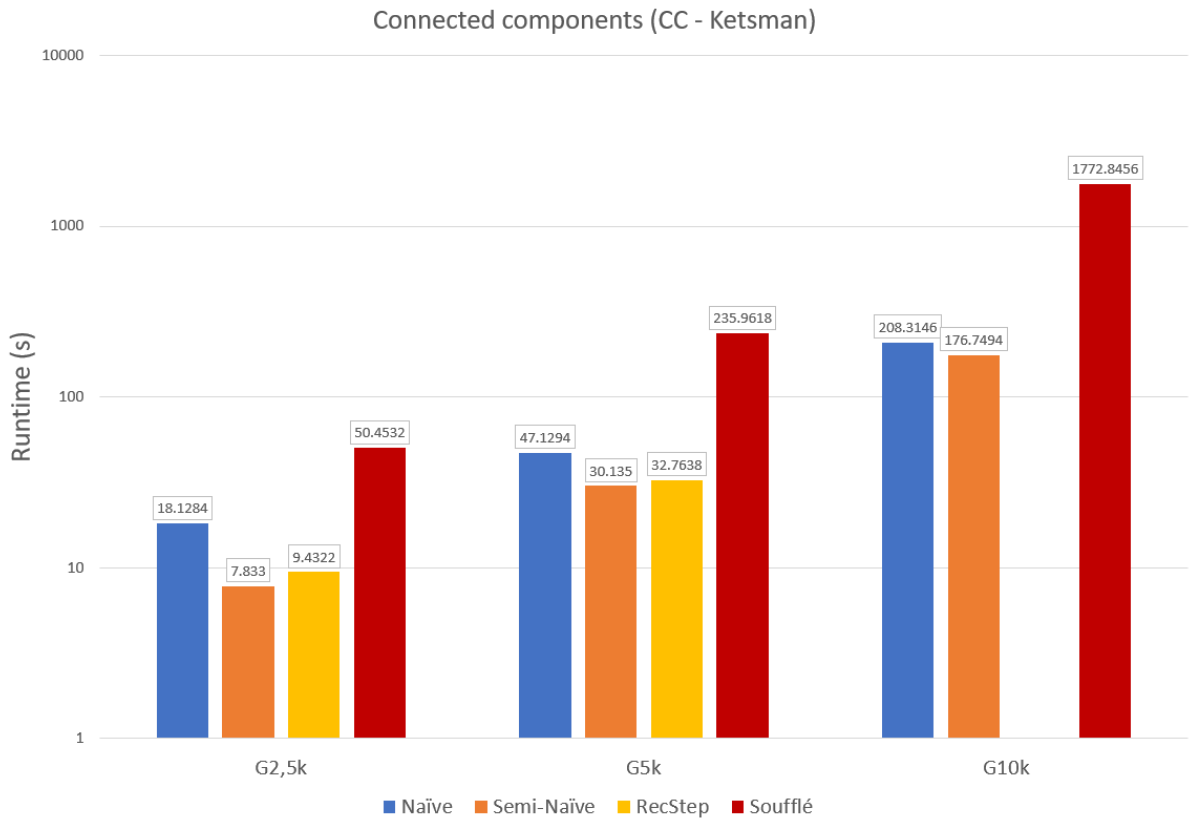

Figure 5.13: Average runtime for every evaluation engine per dataset when evaluating the program for computing connected components (the variant of the program from Bas Ketsman) (figure 5.4).

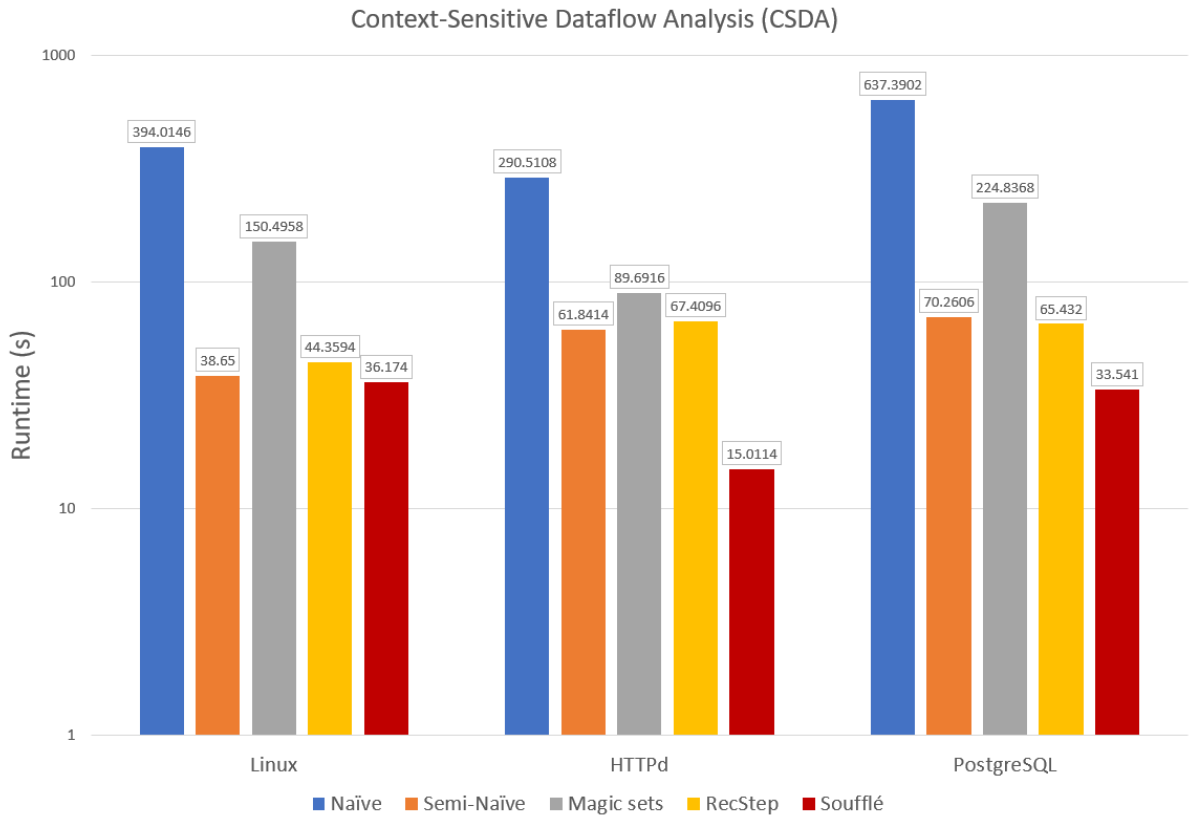

Figure 5.14: Average runtime for every evaluation engine per dataset when evaluating the program for executing context-sensitive dataflow analysis (figure 5.6).

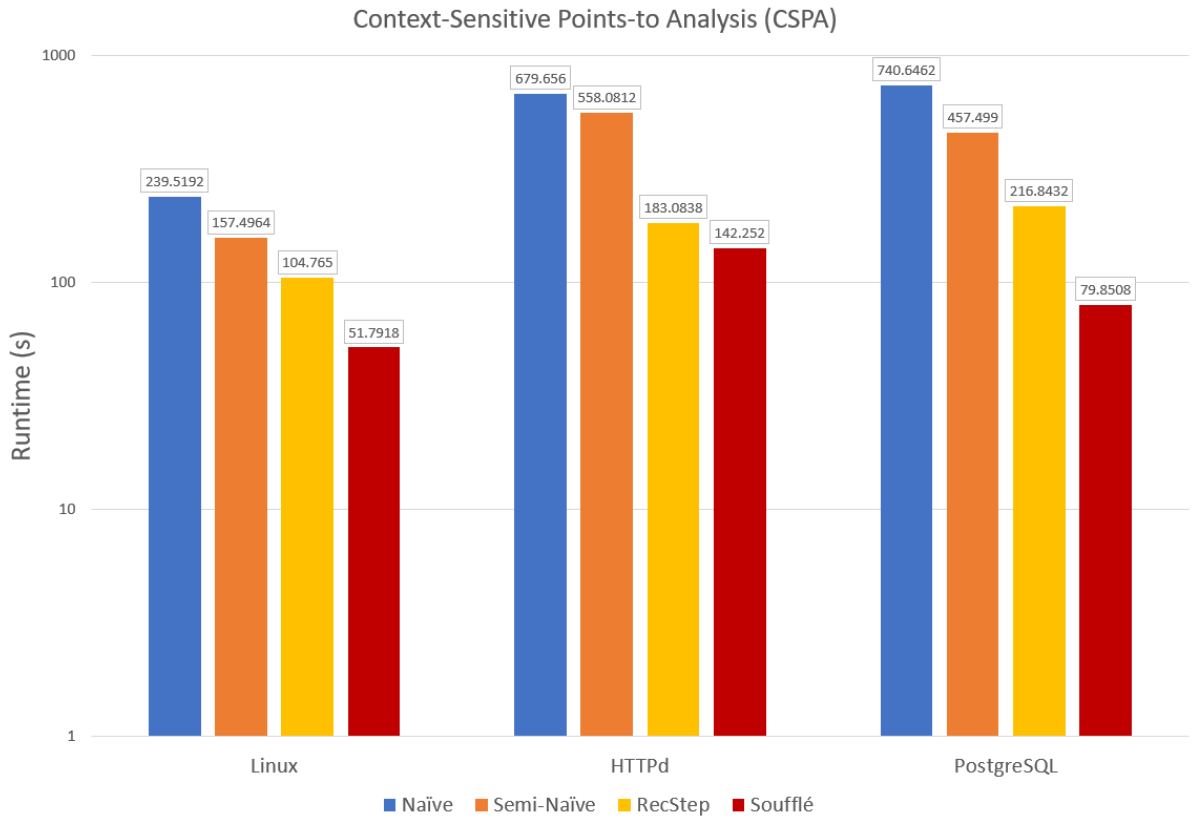

Figure 5.15: Average runtime for every evaluation engine per dataset when evaluating the program for executing context-sensitive points-to analysis (figure 5.5).

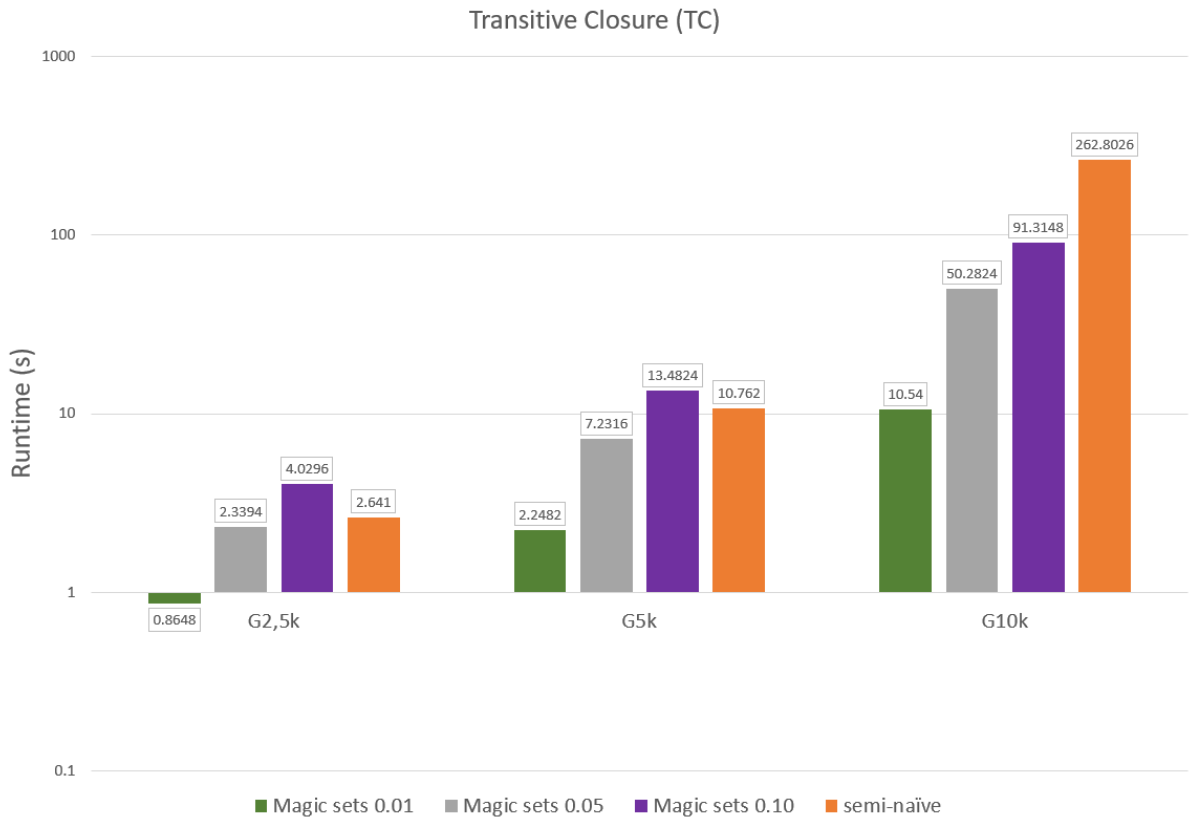

Figure 5.16: Average runtime for the custom implementation using the magic set rewriting technique per dataset when evaluating the program for computing transitive closures (figure 5.1). The decimal numbers are represent the size of the sample when preparing the data file containing the query bindings. The custom implementation using basic semi-naïve evaluation was also added to the graph for easier comparison.

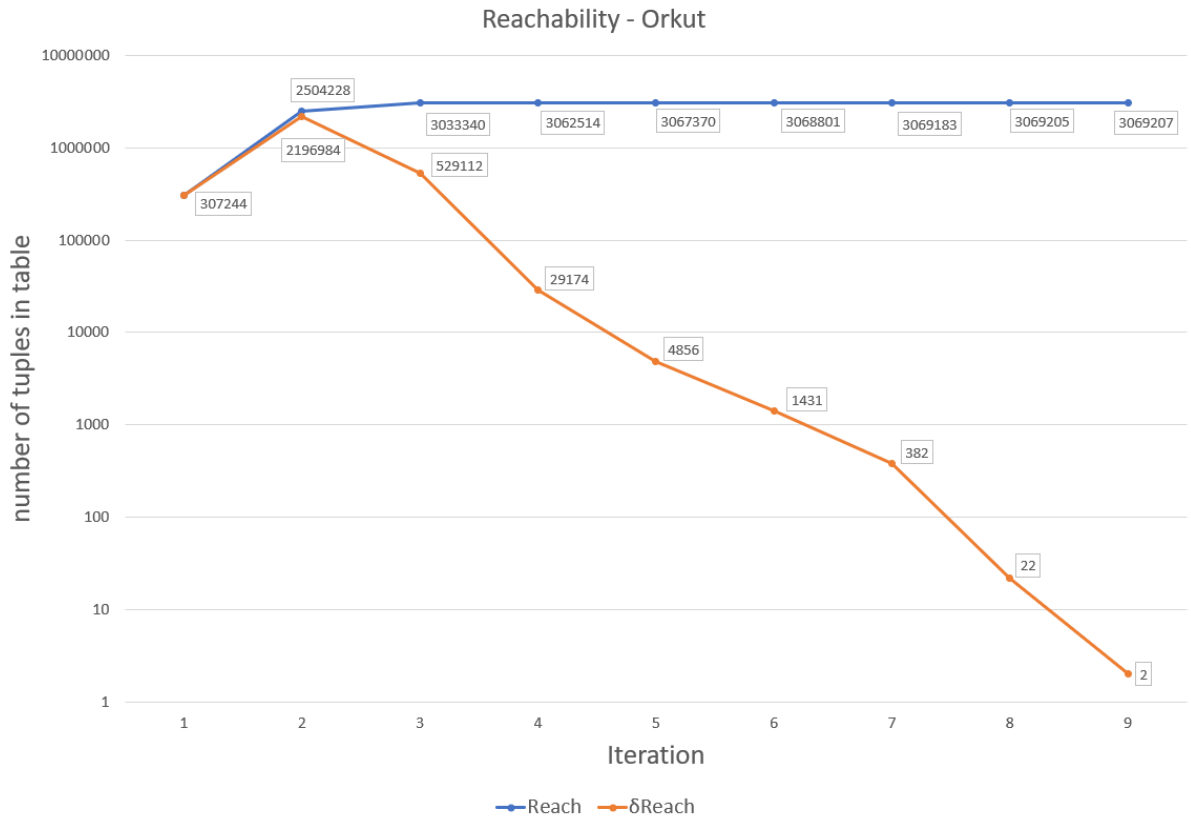

Figure 5.17: The size of the reachability table and the reachability delta table when evaluating the reachability Datalog program on the Orkut dataset.

the significantly better performance of the custom semi-naïve implementation compared to RecStep and Soufflé for Reachability when applied to the LiveJournal and Orkut datasets.

Similar competitiveness of the custom naïve implementation can be found for Ketsman's connected components and the G5k and G10k datasets. Soufflé seems to scale badly because of its inability to optimize on-the-fly, which results in significant degradation because the connected relationship grows quite large.

When we shift our focus to the custom semi-naïve implementation, one thing we notice is that it often possesses the same level of efficiency as RecStep, usually even having a slight edge over RecStep, originating from what we assume to be a difference in overhead. Although we also notice that RecStep seems to scale somewhat better with relation to the size of the datasets. We attribute this to RecStep's flexible on-the-fly optimizations.

It is more difficult to find a consistent pattern when it comes to the comparison between Soufflé and the custom semi-naïve implementation. This is because Soufflé seems to be significantly more dependent on the given Datalog program and dataset. Examples of this can be seen in figures 5.14 and 5.13. When it comes to evaluating the CSDA Datalog program, Soufflé is the best-performing engine across all 3 datasets, albeit with varying leads over the other engines depending on which dataset is observed. On the other hand, for Ketsman's connected components program, Soufflé is always the worst-performing Datalog engine by far.

When we analyze the benchmark results of the semi-naïve evaluation with magic sets rewrite, we observe varied results. For the evaluation of transitive closure, we notice the magic sets evaluation performs best of all techniques overall, with excellent scaling in relation to dataset size. However, this pattern does not repeat for other programs. Let us look at the results for reachability. Here, the magic sets evaluation performs worst of all no matter the dataset. For context-sensitive dataflow analysis, we notice it beats the naïve evaluation in execution time but gets outperformed by semi-naïve, RecStep, and Soufflé (though not by that much for the HTTPd dataset).

Figure 5.18: The number of tuples derived during evaluation of the Datalog program for computing transitive closures on different datasets.

To get a deeper understanding of the measurements for semi-naïve evaluation with magic sets, we also compared the number of derived tuples for all rules in the program between semi-naïve evaluation and semi-naïve evaluation with the magic sets rewrite. The results are visible in figures 5.18, 5.19, and 5.20. When combining these statistics with our measurements, we notice the following pattern: if the difference in the number of derived tuples grows, so does the difference in execution time. For the evaluation of the Datalog program for computing transitive closures, the number of derived tuples is smaller for the semi-naïve evaluation with the magic sets rewrite. Consequently, the execution time is also lower using this technique. On the other hand, when evaluating the Datalog programs for reachability and context-sensitive dataflow analysis, the evaluation using magic sets rewriting derives more tuples, making it slower than normal semi-naïve evaluation.

Though we had hoped for the magic sets evaluation to consistently give a clear improvement, its average to below-average performance is not that surprising. In section 2.2.3 we mentioned SIPS: sideways information passing strategies. These can have quite a significant impact on the efficiency of your evaluation. The custom implementation of the magic sets rewrite technique implements only one optimization over the most basic version of the magic sets rewrite technique, this being the fact that a combination of a supplementary relation and a predicate can be replaced by the subsequent supplementary relation. Apart from this, no sideways information passing strategies came up when applying our magic sets rewrite. Consequently, the efficiency of this technique can definitely be improved compared to the benchmark results measured for this thesis.

Another thing of note for the magic sets evaluation is the query. The query bindings determine how many tuples will be derived. For this study, we arbitrarily chose 5 percent to be the sampling size to acquire our query bindings. However, the results for this evaluation technique change significantly if the amount of query bindings differs significantly. This can be observed in figure 5.16. As expected, if the set of query bindings grows smaller, then the evaluation becomes faster. This is natural, since fewer query bindings mean fewer tuples to be derived. The evaluation with magic sets rewriting even seems to scale better in relation to dataset size than the normal semi-naïve evaluation since semi-naïve evaluation takes the longest to execute when applied to the G10k graph.

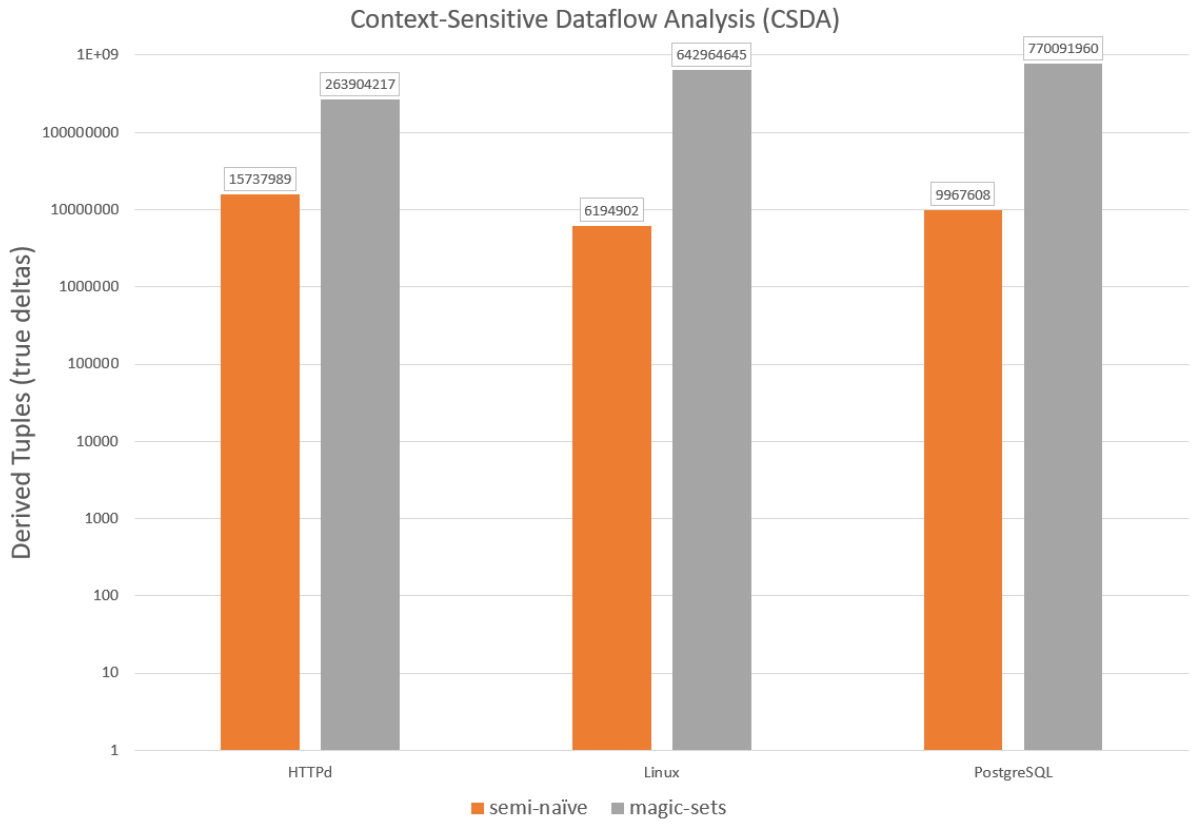

Figure 5.19: The number of tuples derived during evaluation of the Datalog program for executing context-sensitive dataflow analysis on different datasets.

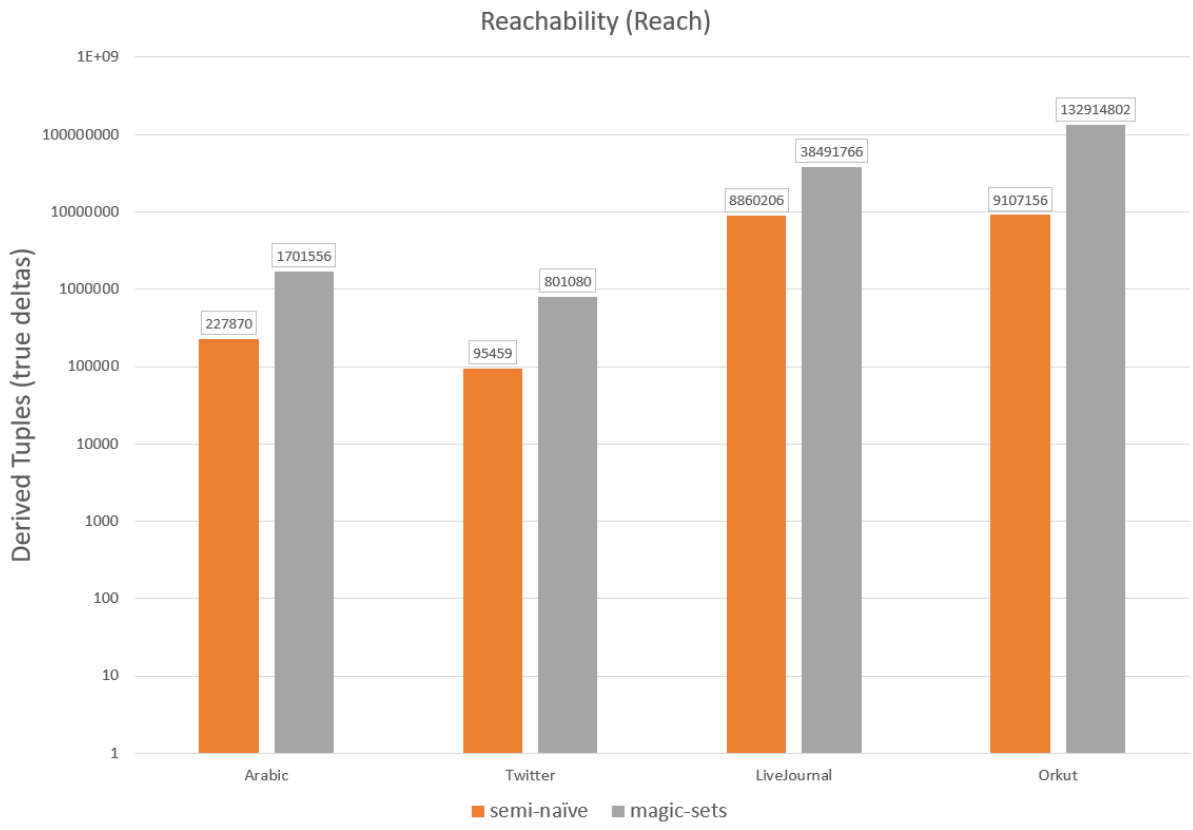

Figure 5.20: The number of tuples derived during evaluation of the Datalog program for computing the reachability of nodes on different datasets.

Chapter 6

Conclusions

Several things can be concluded from our benchmark results, most of which coincide with what we already learned from the theory covered in the earlier part of this thesis. What it boils down to is the following:

- Semi-naïve evaluation engines outperform naïve evaluation engines for any dataset size for which optimization is interesting.
- The magic set rewriting technique is heavily dependent on certain factors:
  - Sideways information passing strategies.
  - Size of the set of query bindings.

This thesis was not able to prove that the magic set rewriting technique combined with semi-naïve evaluation is definitively superior to normal semi-naïve evaluation of a Datalog program. However, the opposite is also not a given. When applying the magic set rewriting technique, it is important to carefully consider the characteristics of the program, the data, and the given queries. The wisest choice is to perform testing before applying this technique in a professional setting.

The only real conclusion that we can make is the fact that further research into the topic is necessary. There are many directions future research can explore. Some possible directions are the following:

- Research into certain sideways information passing strategies, how much of an effect these have on the efficiency of the resulting program and how to implement these sideways information passing strategies efficiently.
- Research into the possible integration of the magic set rewriting technique into existing engines.
- Research into the use of the magic set rewriting technique when combined with more complex Datalog variants. In other words, how does the magic set rewriting technique behave/perform when programs include negation, aggregation, arithmetic, ...

Personal experience with the making of this thesis
I have learned many things over the course of this thesis. One such thing is that I need to work more ahead of time. Otherwise, there might not be enough margin left later on for unforeseen problems. I was able to fix most problems I encountered, but this did cut into the time I had planned to spend on other things (e.g. I was planning to start the writing process sooner, but work still had to be done for the implementation/testing). It would have been better if I had spent my time more evenly during the school year instead of having no choice but to spend many more hours near the end.

Another thing I learned is that too often I try to write code myself instead of first exploring if any libraries exist that can solve the problem I'm facing for me. Not only can it take a lot of time to solve some of the less interesting problems, but the existing library solutions are probably more optimized and easier to utilize effectively. Though, learning to use another person's work can also have a learning curve attached to it.

Furthermore, I was made to realize that I have to think ahead more. Too often, do I make a simple solution to a problem which I then have to edit later on because of functionality I then need. If I make a

decent architecture immediately, I'll lose less time later on when applying changes and creating extensions of my systems.

Naturally, I have learned a lot about the subject matter as well. Many intricacies and nuances are involved when evaluating a Datalog program, and there are probably many more outside of the niche which I have studied over the course of this thesis.

I believe the results I have delivered in this thesis are satisfactory and reflect a solid effort in addressing the research topic. That said, as with any (academic) endeavor, there is of course room for improvement. Not only in the final product but also in the process that led to its completion.

Bibliography

[1] Green, T. J., Huang, S. S., Loo, B. T., & Zhou, W. (2012). Datalog and recursive query processing. Foundations and Trends in Databases, 5(2), 105–195. https://doi.org/10.1561/1900000017

[2] Immerman, N. (1986). Relational queries computable in polynomial time. Information and Control, 68(1–3), 86–104. https://doi.org/10.1016/s0019-9958(86)80029-8

[3] A. Livchak. Languages for polynomial-time queries. Computer-based modeling and optimization of heat-power and electrochemical objects, 1992. In Russian.

[4] Papadimitriou, C. H. (1985). A note the expressive power of Prolog. Bulletin of the European Association for Theoretical Computer Science, 26, 21–22. https://dblp.uni-trier.de/db/journals/eatcs/eatcs26.html#Papadimitriou85

[5] Vardi, M. Y. (1982). The complexity of relational query languages (Extended Abstract). STOC, 137–146. https://doi.org/10.1145/800070.802186

[6] Abiteboul, S., Hull, R., & Vianu, V. (1994). Foundations of databases. https://ci.nii.ac.jp/ncid/BA24328114

[7] Sudarshan, S., & Ramakrishnan, R. (1991). Aggregation and relevance in deductive databases. Very Large Data Bases, 501–511. http://dblp.uni-trier.de/db/conf/vldb/vldb91.html#SudarshanR91

[8] Balbin, I., & Ramamohanarao, K. (1987). A generalization of the differential approach to recursive query evaluation. The Journal of Logic Programming, 4(3), 259–262. https://doi.org/10.1016/0743-1066(87)90004-5

[9] Francois Bancilhon and Raghu Ramakrishnan. An amateur's introduction to recursive query processing strategies. SIGMOD Rec., 15(2):16–52, 1986.

[10] Mumick, I. S., & Pirahesh, H. (1994). Implementation of magic-sets in a relational database system. ACM SIGMOD Record, 23(2), 103–114. https://doi.org/10.1145/191843.191860

[11] Sereni, D., Avgustinov, P., & De Moor, O. (2008). Adding magic to an optimising datalog compiler. SIGMOD, 553–566. https://doi.org/10.1145/1376616.1376673

[12] Seshadri, P., Hellerstein, J. M., Pirahesh, H., Leung, T. Y. C., Ramakrishnan, R., Srivastava, D., Stuckey, P. J., & Sudarshan, S. (1996). Cost-based optimization for magic. ACM SIGMOD Record, 25(2), 435–446. https://doi.org/10.1145/235968.233360

[13] Shen, W., Doan, A., Naughton, J. F., & Ramakrishnan, R. (2007). Declarative information extraction using datalog with embedded extraction predicates. Very Large Data Bases, 1033–1044. http://pages.cs.wisc.edu/∼anhai/courses/784-fall11/xlog.pdf

[14] Stuckey, P. J., & Sudarshan, S. (1994). Compiling query constraints (extended abstract). PODS. https://doi.org/10.1145/182591.182598

[15] Campagna, D., Sarna-Starosta, B., & Schrijvers, T. (2012). Optimizing Inequality Joins in Datalog with Approximated Constraint Propagation. In Lecture notes in computer science (pp. 108–122). https://doi.org/10.1007/978-3-642-27694-1_9

[16] Kifer, M. (1998). On the decidability and axiomatization of query finiteness in deductive databases. Journal of the ACM, 45(4), 588–633. https://doi.org/10.1145/285055.285058

[17] Ramakrishnan, R., Bancilhon, F., & Silberschatz, A. (1987). Safety of recursive Horn clauses with infinite relations. PODS. ACM., 328–339. https://doi.org/10.1145/28659.28694

[18] Aref, M., Cate, B. T., Green, T. J., Kimelfeld, B., Olteanu, D., Pasalic, E., Veldhuizen, T. L., & Washburn, G. (2015). Design and Implementation of the LogicBlox System. SIGMOD. https://doi.org/10.1145/2723372.2742796

[19] Shkapsky, A., Yang, M., Interlandi, M., Chiu, H., Condie, T., & Zaniolo, C. (2016). Big Data Analytics with Datalog Queries on Spark. Proceedings of the 2022 International Conference on Management of Data. https://doi.org/10.1145/2882903.2915229

[20] Seo, J., Guo, S., & Lam, M. S. (2015). SociaLite: an efficient graph query language based on Datalog. IEEE Transactions on Knowledge and Data Engineering, 27(7), 1824–1837. https://doi.org/10.1109/tkde.2015.2405562

[21] Whaley, J., & Lam, M. S. (2004). Cloning-based context-sensitive pointer alias analysis using binary decision diagrams. Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation, 131–144. https://doi.org/10.1145/996841.996859

[22] Scholz, B., Jordan, H., Subotić, P., & Westmann, T. (2016). On fast large-scale program analysis in Datalog. CC. ACM., 196–206. https://doi.org/10.1145/2892208.2892226

[23] Fan, Z., Zhu, J., Zhang, Z., Albarghouthi, A., Koutris, P., & Patel, J. M. (2019). Scaling-up in-memory datalog processing. Proceedings of the VLDB Endowment, 12(6), 695–708. https://doi.org/10.14778/3311880.3311886

[24] Wang, K., Hussain, A., Zuo, Z., Xu, G., & Sani, A. A. (2017). Graspan. ACM SIGARCH Computer Architecture News, 45(1), 389–404. https://doi.org/10.1145/3093337.3037744

[25] Wolfson, O., & Silberschatz, A. (1988). Distributed processing of logic programs. ACM SIGMOD Record, 17(3), 329–336. https://doi.org/10.1145/971701.50242

[26] Wang, J., Balazinska, M., & Halperin, D. (2015). Asynchronous and fault-tolerant recursive datalog evaluation in shared-nothing engines. Proceedings of the VLDB Endowment, 8(12), 1542–1553. https://doi.org/10.14778/2824032.2824052

[27] Yang, M., Shkapsky, A., & Zaniolo, C. (2016). Scaling up the performance of more powerful Datalog systems on multicore machines. The VLDB Journal, 26(2), 229–248. https://doi.org/10.1007/s00778-016-0448-z

[28] Martínez-Angeles, C. A., Dutra, I., Costa, V. S., & Buenabad-Chávez, J. (2014). A datalog engine for GPUs. In Lecture notes in computer science (pp. 152–168). https://doi.org/10.1007/978-3-319-08909-6_10

[29] Ameloot, T. J., Neven, F., & Van Den Bussche, J. (2013). Relational transducers for declarative networking. Journal of the ACM, 60(2), 1–38. https://doi.org/10.1145/2450142.2450151

[30] Ameloot, T. J., Ketsman, B., Neven, F., & Zinn, D. (2015). Weaker forms of monotonicity for declarative networking. ACM Transactions on Database Systems, 40(4), 1–45. https://doi.org/10.1145/2809784

[31] Jordan, H., Subotić, P., Zhao, D., & Scholz, B. (2019). A specialized B-tree for concurrent datalog evaluation. PPoPP. ACM., 327–339. https://doi.org/10.1145/3293883.3295719

[32] Afrati, F. N., & Ullman, J. D. (2012). Transitive closure and recursive Datalog implemented on clusters. EDBT. ACM., 132–143. https://doi.org/10.1145/2247596.2247613

[33] Rudolph, S. & M. Thomazo. (2016). “Expressivity of Datalog Variants - Completing the Picture”. In: IJCAI. IJCAI/AAAI Press. 1230–1236.

[34] Ketsman, B., & Koch, C. (2020). Datalog with Negation and Monotonicity. International Conference on Database Theory, 18. https://doi.org/10.4230/lipics.icdt.2020.19

[35] Sudarshan, S., & Ramakrishnan, R. (1991b). Aggregation and relevance in deductive databases. Very Large Data Bases, 501–511. http://dblp.uni-trier.de/db/conf/vldb/vldb91.html#SudarshanR91

[36] Zaniolo, C., Yang, M., Das, A., & Interlandi, M. (2016). The Magic of Pushing Extrema into Recursion: Simple, Powerful Datalog Programs. AMW. http://ceur-ws.org/Vol-1644/paper16.pdf

[37] Zaniolo, C., Yang, M., Das, A., Shkapsky, A., Condie, T., & Interlandi, M. (2017). Fixpoint semantics and optimization of recursive Datalog programs with aggregates. Theory and Practice of Logic Programming, 17(5–6), 1048–1065. https://doi.org/10.1017/s1471068417000436

[38] Wang, Y. R., Khamis, M. A., Ngo, H. Q., Pichler, R., & Suciu, D. (2022). Optimizing Recursive Queries with Progam Synthesis. Proceedings of the 2022 International Conference on Management of Data, 79–93. https://doi.org/10.1145/3514221.3517827

[39] Jachiet, L., P. Genevès, N. Gesbert, and N. Layaïda. (2020). "On the Optimization of Recursive Relational Queries: Application to Graph Queries". In: SIGMOD Conference. ACM. 681–697.

[40] Ketsman, B., & Koutris, P. (2022). Modern datalog engines. https://doi.org/10.1561/9781638280439

[41] Fan, Z., Zhu, J., Zhang, Z., Albarghouthi, A., Koutris, P., & Patel, J. M. (n.d.). GitHub - Hacker0912/RecStep. GitHub. https://github.com/Hacker0912/RecStep

[42] Fan, Z., Zhu, J., Zhang, Z., Albarghouthi, A., Koutris, P., & Patel, J. M. (2019c). Scaling-up in-memory datalog processing. Proceedings of the VLDB Endowment, 12(6), 695–708. https://doi.org/10.14778/3311880.3311886

[43] Wang, K., Hussain, A., Zuo, Z., Xu, G., & Sani, A. A. (2017b). Graspan. ACM SIGARCH Computer Architecture News, 45(1), 389–404. https://doi.org/10.1145/3093337.3037744

[44] Bader, David A., & Kamesh Madduri. "GTgraph: A Suite of Synthetic Random Graph Generators." Psu.edu, 2025, www.cse.psu.edu/ kxm85/software/GTgraph

[45] Wang, K., Hussain, A., Zuo, Z., Xu, G., & Sani, A. A. "Graphs - Google Drive." Google Drive, 2024, drive.google.com/drive/folders/1DRj-cfISV9v34vU7DH1PKEu13PaMIf71.

[46] Abadi, M., & Loo, B. T. (2007). Towards a declarative language and system for secure networking. NetDB., 2.

[47] Abiteboul, S., Bienvenu, M., Galland, A., & Antoine, É. (2011). A rule-based language for web data management. HAL, 293–304. https://doi.org/10.1145/1989284.1989320

[48] Alvaro, P., Marczak, W. R., Conway, N., Hellerstein, J. M., Maier, D., & Sears, R. (2011). Dedalus: Datalog in time and space. In Lecture notes in computer science (pp. 262–281). https://doi.org/10.1007/978-3-642-24206-9_16

[49] Subotić, P., Jordan, H., Chang, L., Fekete, A., & Scholz, B. (2018). Automatic index selection for large-scale datalog computation. Proceedings of the VLDB Endowment, 12(2), 141–153. https://doi.org/10.14778/3282495.3282500

[50] Lamb, A., Shen, Y., Heres, D., Chakraborty, J., Kabak, M. O., Hsieh, L., & Sun, C. (2024). Apache Arrow DataFusion: A Fast, Embeddable, Modular Analytic Query Engine. SIGMOD, 5–17. https://doi.org/10.1145/3626246.3653368